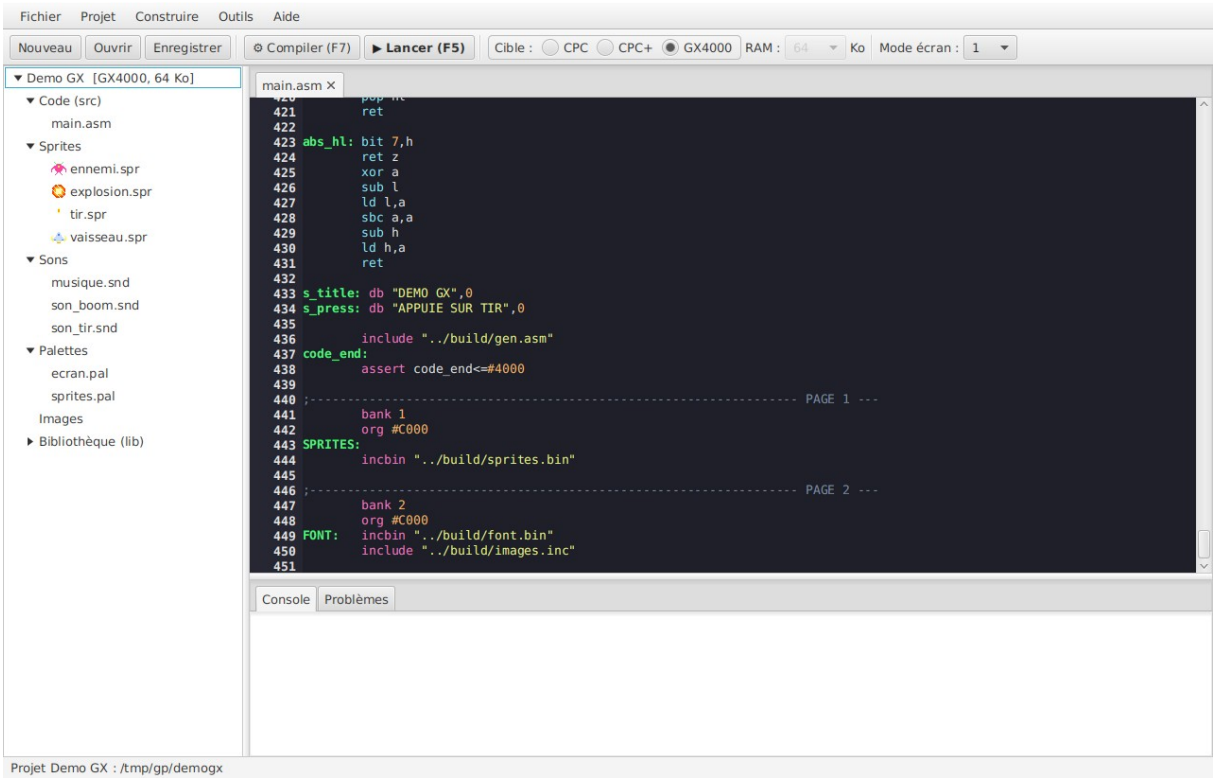
GX Studio

Atelier de création de jeux pour Amstrad CPC, CPC+ et GX4000

Java / JavaFX sous Linux — assembleur rasm — émulateur Caprice32 — version 0.5.2

(c) 2026 ALONSO Stéphane [Fanou]



La fenêtre principale : projet, éditeur de code, console

GX Studio regroupe dans une seule application tout ce qu'il faut pour écrire un jeu en assembleur Z80 pour les Amstrad : un éditeur de code, des éditeurs de sprites, de palettes et de sons, la conversion automatique de ces ressources, la compilation par **rasm** et le lancement dans **Caprice32** avec les points d'arrêt posés dans l'éditeur. Chaque projet repose sur une bibliothèque de routines testée sur une vraie GX4000 (celle de STELLAR WINGS pour les CPC+, et une bibliothèque équivalente pour le CPC classique).

1. Installation

Élément	Rôle	Installation (Debian / Ubuntu)
Java 17 ou plus	exécute GX Studio	sudo apt install openjdk-21-jdk
JavaFX	interface graphique	sudo apt install openjfx (ou SDK JavaFX : variable JAVAFX)
RichTextFX	éditeur de code coloré	fourni : libs/richtextfx-fat.jar (ou Maven)

rasm	assembleur Z80, produit .cpr et .dsk	github.com/EdouardBerge/rasm
Caprice32	émulateur CPC / CPC+, débogueur	voir caprice32/LISEZMOI.txt (correctif conseillé)
SDCC 4.x (facultatif)	compilateur C pour Z80 (projets en C, chapitre 6)	sudo apt install sdcc
Python 3 (facultatif)	scripts personnels	non nécessaire : la conversion est en Java

```
./build.sh              compilation sans Maven (javac + jar) -> out/gxstudio.jar
./gxstudio.sh           lancement de l'interface
mvn package             ou avec Maven (JavaFX et RichTextFX téléchargés)
mvn javafx:run
```

Au premier lancement, **Outils > Réglages** : indiquer le chemin de rasm et de cap32 (et éventuellement des options Caprice32 comme `-O system.limit_speed=1`). Les réglages sont dans ~/.gxstudio/settings.properties.

GX Studio lance Caprice32 depuis le dossier de l'exécutable et lui indique le dossier **rom/** qui s'y trouve (ROM du 464, 664, 6128 et AMSDOS). Si un projet CPC fait s'ouvrir puis se fermer aussitôt l'émulateur, la console affiche « Couldn't open ROM file » : ajouter dans les options des Réglages `-O rom.rom_path=/chemin/vers/caprice32/rom` (ou régler rom_path dans la section [rom] de cap32.cfg). Les cibles CPC+ et GX4000 ne sont pas concernées : la cartouche contient sa propre ROM.

Le correctif Caprice32

Le fichier caprice32/gxstudio-caprice32.patch (à appliquer sur Caprice32, voir caprice32/LISEZMOI.txt) apporte deux choses : la fidélité aux CPC+ et la liaison avec le studio.

Correction / ajout	Pourquoi
Interruptions raster (PRI) sur 8 bits	le correctif compare 8 bits du numéro de ligne (9 bits avec CAP32_PRI_9BITS=1) ; la bibliothèque mesure le comportement au démarrage et marche dans les deux cas
Sprites mémorisés à chaque ligne	un sprite déplacé pendant l'image (multiplexage) s'affiche plusieurs fois, comme sur la console
6128+ en 64 Ko	= GX4000
Soft-scroll vertical : la rangée suivante boucle sur 2 Ko	défilement 8 directions (scroll8.asm) juste sur toute la mémoire écran
Sprite sous le bas de l'écran	le correctif 0.3 pouvait faire planter Caprice32 dans ce cas : corrigé
Serveur de débogage (devtools.server_port)	pause, pas à pas, registres, mémoire, points d'arrêt pilotés depuis le studio (chapitre 4)
Profil d'exécution (0.5.1)	cycles du Z80 comptés instruction par instruction : Débogage > Mesurer les performances (4.2)
Points d'arrêt testés avant l'instruction (0.5.1)	un point d'arrêt pouvait être manqué quand une fin d'image tombait juste avant son adresse

Sans le correctif, les jeux CPC classiques fonctionnent normalement, mais les rasters et le multiplexage des projets CPC+ ne s'affichent pas comme sur la console, et le débogage se fait dans les outils de Caprice32 (Maj+F2) au lieu du studio.

Dépannage de l'installation

Message	Cause et solution
error: release version 17 not supported	le javac du PATH est plus ancien que le java (ex. JDK 11 + JDK 23).

	build.sh prend maintenant le javac situé à côté de java ; sinon : sudo update-alternatives --config javac
error: module not found: javafx.controls	JavaFX absent : sudo apt install openjfx, ou SDK JavaFX de Gluon avec export JAVAFX=<sdk>/lib
Graphics Device initialization failed for : es2, sw / no suitable pipeline found	bibliothèques natives de JavaFX introuvables (JDK d'Oracle + paquet openjfx). gxstudio.sh ajoute /usr/lib/x86_64-linux-gnu/jni (variable JNI) ; le SDK JavaFX de Gluon les contient déjà
rasm introuvable / Caprice32 introuvable	chemins à indiquer dans Outils > Réglages

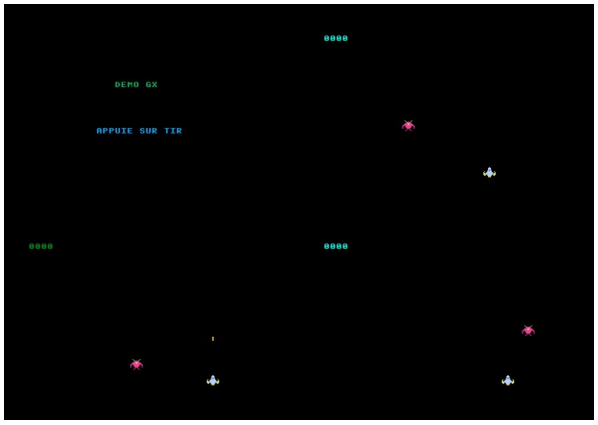
Pour un premier jeu pas à pas, voir le **tutoriel du casse-brique GX4000** (Tutoriel_casse_brique_GX.pdf) : création du projet, sprites, sons, code commenté, rasters, débogage et copie sur la console.

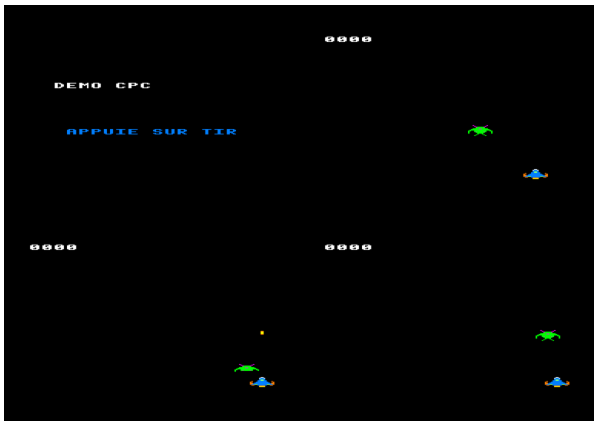
2. Créer un projet

Fichier > Nouveau projet : nom du jeu, dossier, machine, mémoire, mode écran et **langage** (assembleur Z80, ou C avec SDCC et assembleur, voir le chapitre 6). La machine se choisit aussi à tout moment dans la barre d'outils (CPC, CPC+, GX4000).

Cible	Sortie	Bibliothèque	Mémoire proposée	Particularités
CPC 464/664/6128	disquette .dsk (RUN"NOM)	lib/cpc + lib/common	64, 128, 256, 512, 576 Ko	sprites logiciels mode 0, son par interruption, clavier + manettes
CPC 464+/6128+	cartouche .cpr	lib/plus + lib/common	64, 128 Ko	sprites matériels, rasters, son DMA, clavier + manettes
GX4000	cartouche .cpr	lib/plus + lib/common	64 Ko	pas de clavier : HAS_KEYBOARD = 0, manettes seulement

Le projet de départ est un petit jeu complet et commenté : un vaisseau dirigé à la manette tire sur un ennemi qui rebondit, avec une musique, deux bruitages et un score. Il compile et se lance tout de suite (F5) ; on le transforme ensuite en son propre jeu.





Le projet de départ : GX4000 (sprites matériels, dégradé raster) et CPC (sprites logiciels)

Dossier / fichier	Contenu
projet.gxs	nom, cible, mémoire, mode écran, fichier principal, définitions -D
src/main.asm	votre programme (point de départ fourni) ; d'autres .asm peuvent être ajoutés
lib/	bibliothèque copiée depuis les modèles (Projet > Mettre à jour la bibliothèque)
assets/sprites/*.spr	sprites (texte : un caractère par pixel)
assets/palettes/ecran.pal, sprites.pal	palettes (16 couleurs #RGB)
assets/sons/*.snd	musiques et bruitages
assets/images/*.png	images converties dans le mode écran du projet
build/	fichiers générés : config.inc, gen.asm, *.bin, .cpr/.dsk, symboles (ne pas modifier)
.breakpoints	points d'arrêt posés dans l'éditeur

3. L'interface

Aide > À propos présente le studio et son auteur.

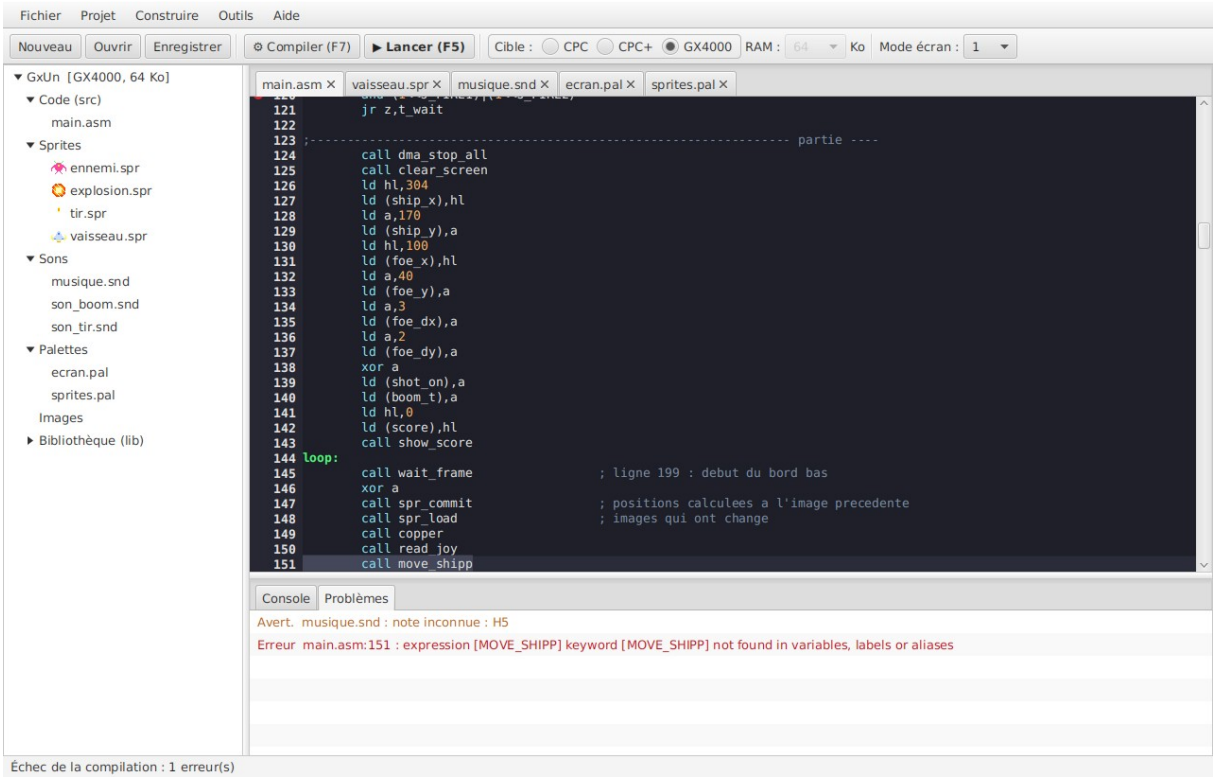


La boîte « À propos »

3.1 Le code

L'éditeur colore les instructions Z80, les directives rasm, les étiquettes, les nombres et les commentaires. Un clic dans la marge (ou F9) pose un **point d'arrêt** ; il suit sa ligne quand on

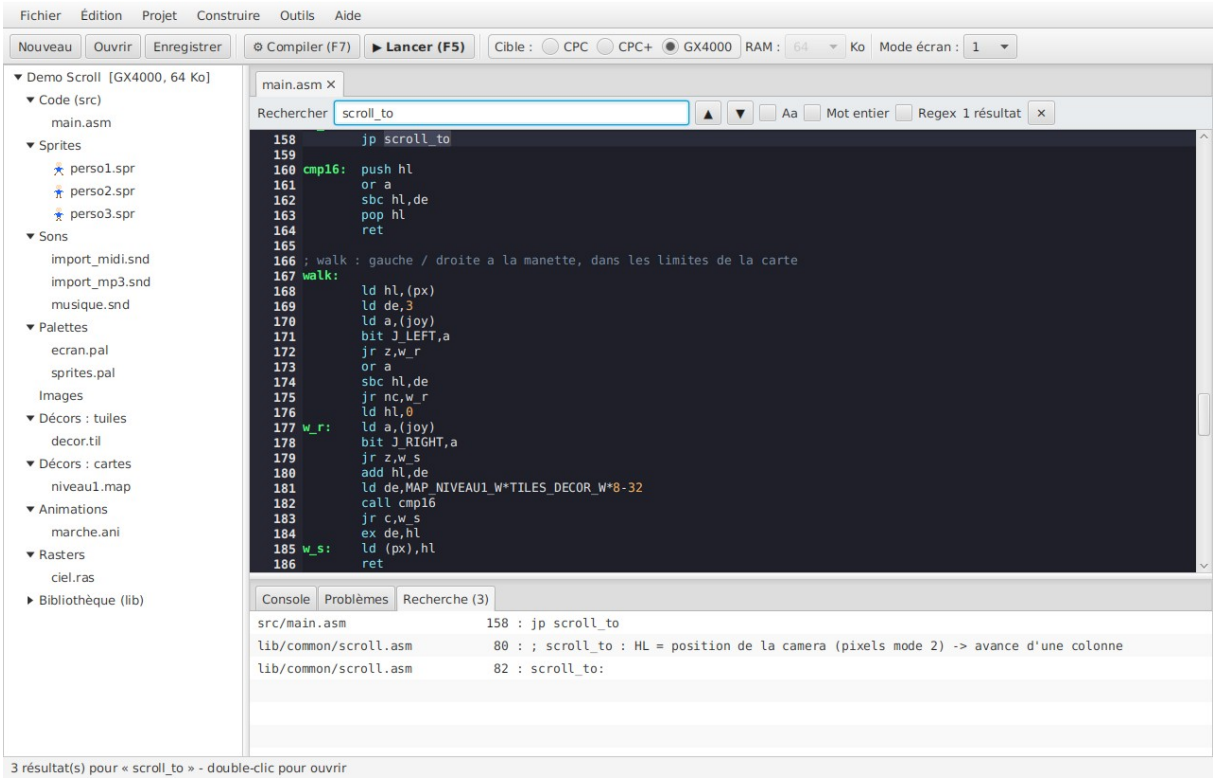
insère ou supprime des lignes au-dessus. **F7** compile, **F5** compile et lance Caprice32. Les erreurs de rasm apparaissent dans l'onglet Problèmes : un double-clic ouvre le fichier à la bonne ligne (la première erreur est ouverte automatiquement).



Une erreur de compilation et une note inconnue dans une musique

Le menu **Édition** offre la recherche dans le code :

Commande	Raccourci	Effet
Rechercher	Ctrl+F	barre de recherche au-dessus du code (texte sélectionné repris) : toutes les occurrences sont surlignées, Entrée / Maj+Entrée = suivante / précédente ; options majuscules, mot entier, expression régulière
Remplacer	Ctrl+H	même barre avec le champ de remplacement : Remplacer (une à une) ou Tout remplacer
Suivant / Précédent	F3 / Maj+F3	occurrence suivante ou précédente
Rechercher dans le projet	Ctrl+Maj+F	tous les fichiers .asm / .inc (src, lib et fichiers générés) ; résultats dans l'onglet Recherche, double-clic pour ouvrir à la bonne ligne
Aller à la définition	F12	ouvre l'endroit où est définie l'étiquette sous le curseur (étiquette:, equ, VAR, macro), même dans la bibliothèque ou dans build/gen.asm
Aller à la ligne	Ctrl+G	saisir un numéro de ligne



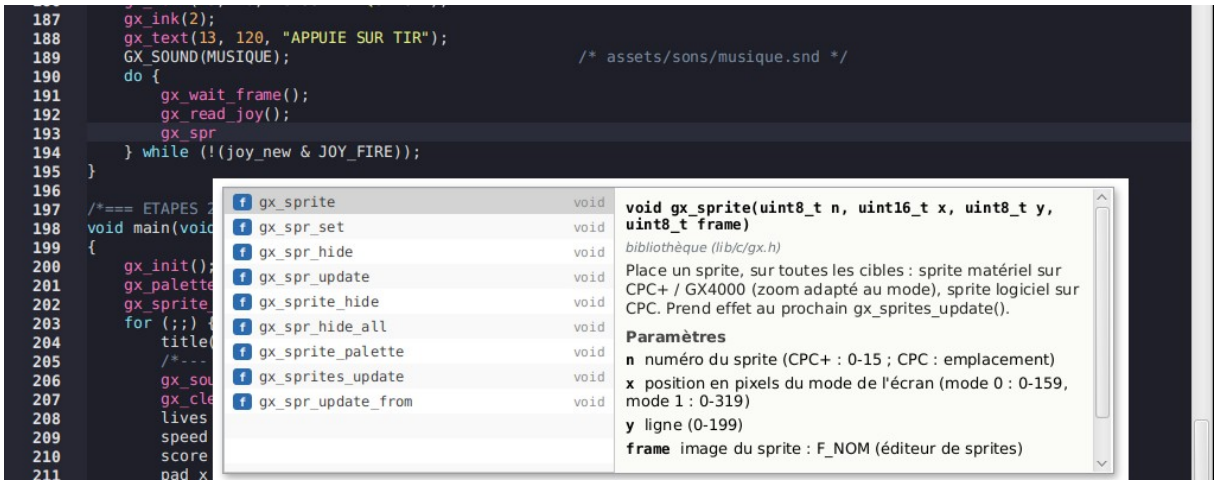
Recherche dans le code (Ctrl+F) et dans tout le projet (onglet Recherche)

3.1 bis Complétion et aide sur la bibliothèque

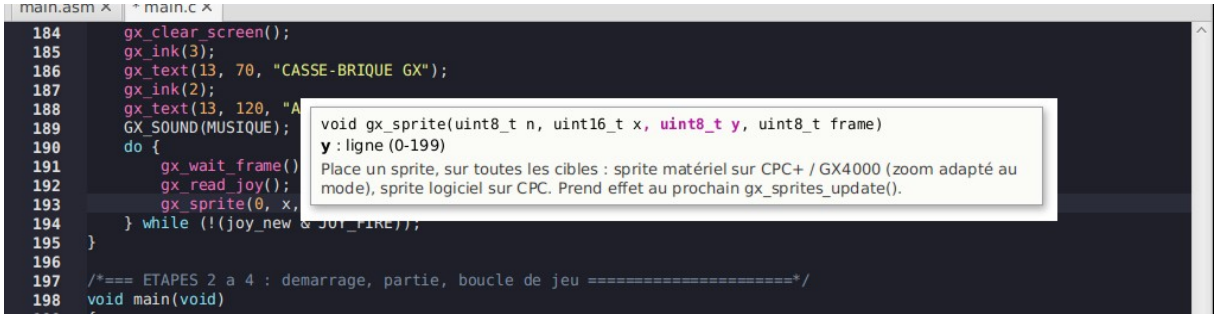
Comme dans un IDE moderne, l'éditeur aide à écrire le code (depuis la version 0.5.2) :

Commande	Raccourci	Effet
Compléter le code	Ctrl+Espace	liste des noms qui commencent par (ou contiennent) le début du mot : fonctions **gx_…** (C) ou routines (asm) de la bibliothèque, ressources du projet (F_…, MAP_…, TILES_…, sons…), étiquettes et fonctions du projet, instructions Z80, directives rasm, mots du C. On continue à taper pour filtrer ; ↑ ↓ pour choisir, **Entrée** ou **Tab** pour insérer, Échap pour fermer. S'il n'y a qu'un choix, il est inséré directement.
Paramètres	(ou , — Ctrl+Maj+Espace	en C, une bulle au-dessus de la ligne montre la signature de la fonction appelée, le paramètre en cours en couleur et son explication
Aide du mot	Ctrl+Maj+Espace, ou souris arrêtée sur le mot	aide de la fonction, de la routine ou de la constante sous le curseur (aussi en assembleur)

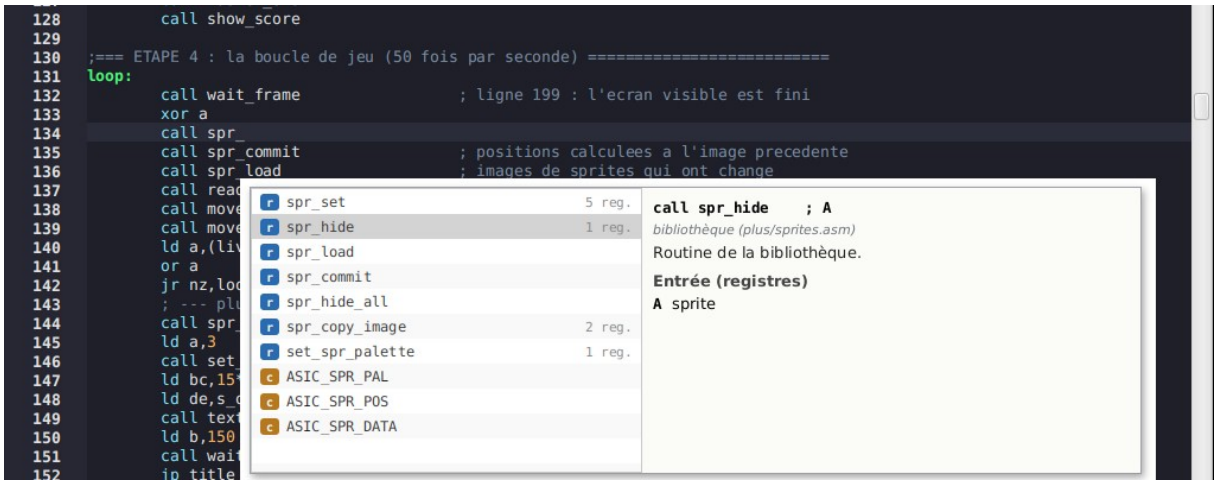
À droite de la liste, la documentation du nom choisi : signature, cible (certaines fonctions n'existent que sur CPC+ / GX4000), résumé, paramètres (en C) ou registres d'entrée et de sortie (en assembleur), et un exemple. Une fonction C insérée reçoit ses parenthèses, le curseur se place entre elles et la bulle des paramètres s'ouvre. Dans les paramètres d'un appel, les fonctions qui ne renvoient rien (void) passent en fin de liste ; après `call` ou `jp`, les routines et étiquettes passent devant les instructions.



Ctrl+Espace dans un projet C : les fonctions gx_spr… et leur documentation



La bulle des paramètres : le paramètre en cours (y) est en couleur



En assembleur : routines de la bibliothèque et registres d'entrée

La documentation de l'API C est intégrée au studio. Celle des routines assembleur est lue dans le commentaire qui précède chaque routine de lib/ : `; nom : A = …, HL = … -> A = …` (registres d'entrée, puis après la flèche ceux de sortie, puis une remarque). Une routine ajoutée à lib/ ou à src/ avec un commentaire de cette forme apparaît donc automatiquement, documentée ; pour les étiquettes et fonctions du projet, c'est le commentaire juste au-dessus qui sert de résumé.

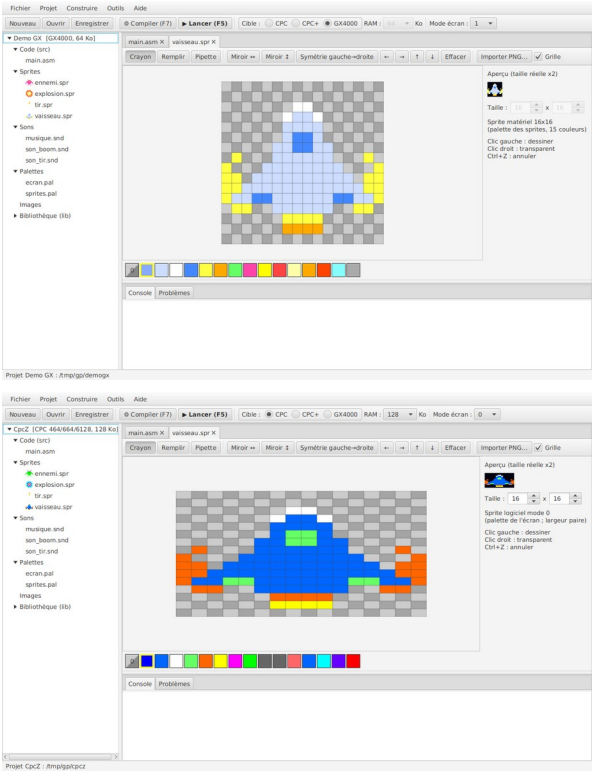
Le programme principal inclut les fichiers générés : `build/config.inc` (en tête) et `build/gen.asm`. config.inc décrit la cible pour que le même code puisse s'adapter :

```
TARGET_CPC      equ 0
TARGET_PLUS     equ 0
TARGET_GX4000   equ 1
ASIC            equ 1               ; sprites materiels, rasters, DMA
HAS_KEYBOARD    equ 0
SOUND_SW        equ 0               ; 1 = lecteur de son logiciel
RAM_KB          equ 64
SCREEN_MODE     equ 1
```

3.2 Les sprites

L'éditeur de sprites dessine au pixel : clic gauche avec l'encre choisie, clic droit pour le transparent, outils Remplir et Pipette, miroirs, symétrie gauche→droite (pratique pour les vaisseaux), décalages, annulation (Ctrl+Z) et import d'une image PNG (chaque pixel prend l'encre la plus proche). L'aperçu montre le sprite avec les proportions réelles de la machine.

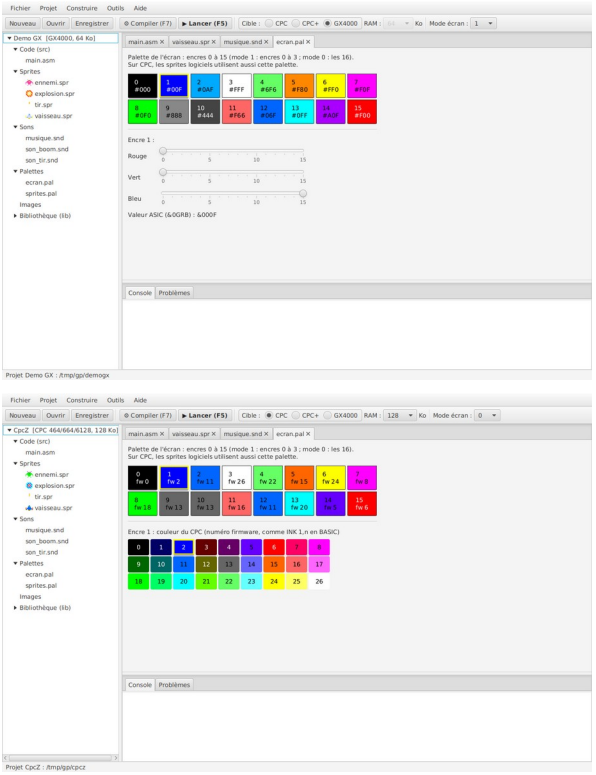
- **CPC+ / GX4000** : sprites matériels de 16 pixels de large, couleurs 1 à 15 de la palette des sprites. Un sprite plus haut que 16 lignes occupe plusieurs images 16 x 16 empilées (`F_NOM` = la première, `H_NOM` = leur nombre). Les images sont rangées dans `build/sprites.bin`, normalement en page 1 de la cartouche (64 images au plus par page). Toutes cibles : `build/sprites_m0.bin` contient aussi chaque sprite en octets mode 0 (`M0_NOM` = sa position), pour le dessiner dans le décor.
- **CPC** : sprites logiciels en mode 0, de taille libre (largeur paire), dans les encres de l'écran. Le studio produit pour chaque sprite un masque et les données (`spr_nom`, table `spr_table`), dessinés par lib/cpc/swsprites.asm qui sauvegarde et restaure le décor.



Le même sprite en matériel (GX4000) et en logiciel mode 0 (CPC)

3.3 Les palettes

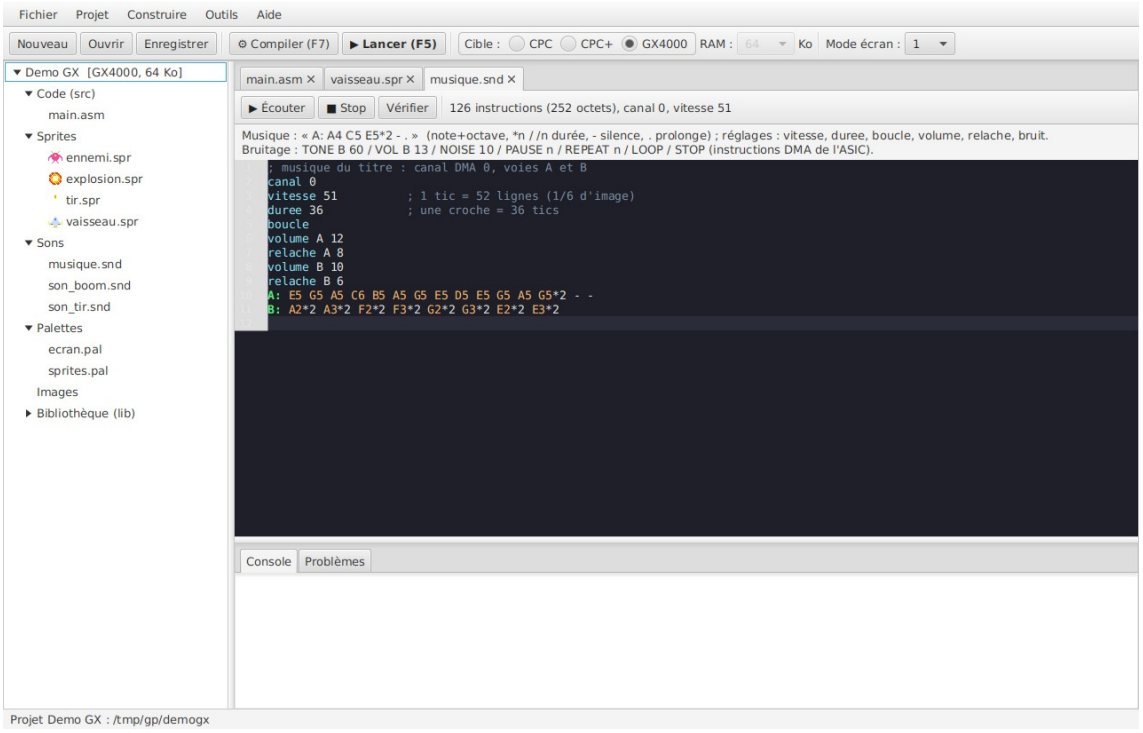
Sur CPC+ et GX4000, chaque encre se règle en rouge, vert et bleu de 0 à 15 (4096 couleurs) ; la valeur ASIC &0GRB est affichée. Sur CPC, on choisit parmi les 27 couleurs du Gate Array, numérotées comme en BASIC (INK n,couleur). Les palettes sont stockées en #RGB : passer un projet de CPC+ à CPC ramène chaque couleur à la plus proche des 27.



Palette CPC+ (4096 couleurs) et palette CPC (27 couleurs)

3.4 Les sons

Un son est un petit texte. Le bouton **Écouter** le joue immédiatement grâce à une émulation du PSG intégrée au studio, sans lancer Caprice32. Le studio le compile en **liste DMA de l'ASIC** : sur CPC+/GX4000 le DMA la joue seul ; sur CPC, la bibliothèque contient un lecteur logiciel qui interprète exactement les mêmes listes (appelé 300 fois par seconde par l'interruption). Une musique écrite pour une machine fonctionne donc sur l'autre.



Éditeur de son : coloration, vérification et écoute

Ligne	Effet

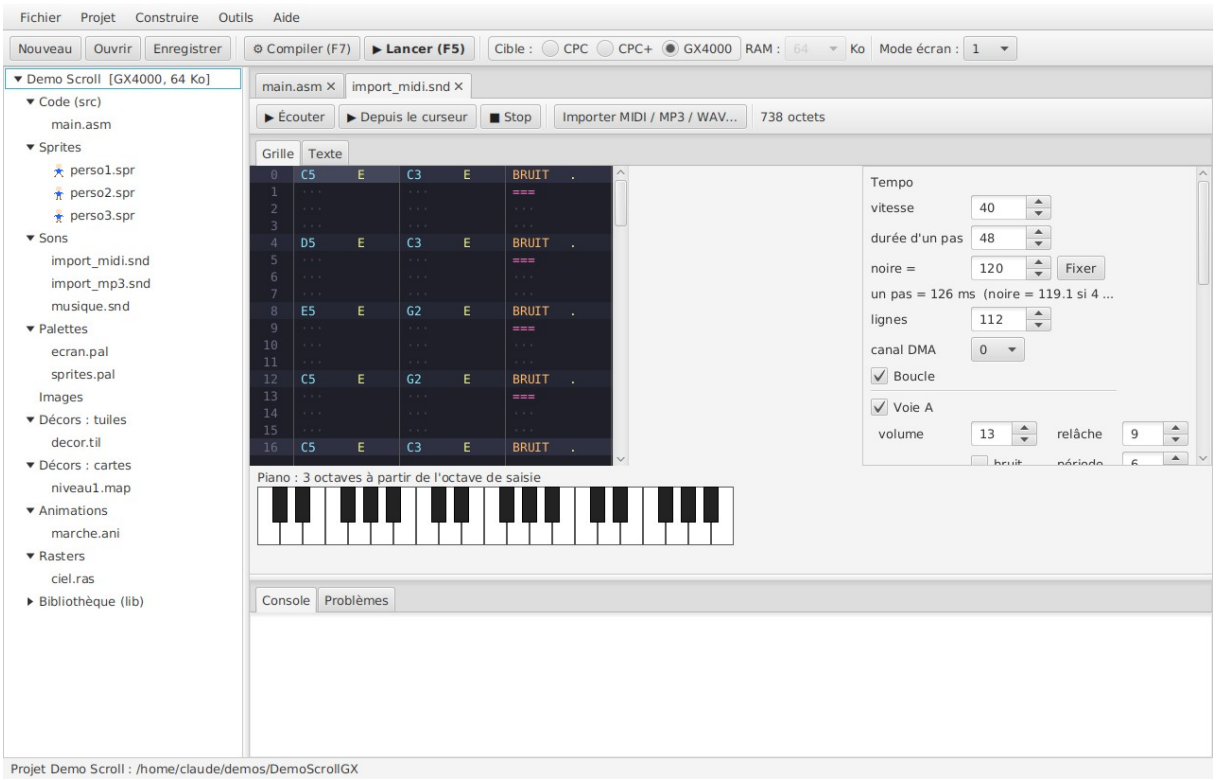
canal 0	canal à utiliser (0-2) : équate NOM_CH
vitesse 51	diviseur : 1 tic = vitesse + 1 lignes (51 = 1/6 d'image) : équate NOM_SPEED
duree 36	durée d'une note en tics
boucle	la musique recommence sans fin
volume A 12 / relache A 8	volume de la voie ; volume après les 2/3 de la note (0 = tenu)
bruit C 10	la voie C joue du bruit (période 10) au lieu de notes
A: E5 G5 A5*2 - G5/2 .	notes (note + octave, # pour dièse), *n ou /n sur la durée, - silence, . prolonge
TONE B 60, VOL B 13, NOISE 10	bruitage : écriture directe dans le PSG
PAUSE n, REPEAT n, LOOP, STOP	instructions DMA (une pause dure n x (vitesse + 1) lignes)

```
; jouer la musique (CPC+ / GX4000)
        ld a,MUSIQUE_CH
        ld hl,DMA_RAM+MUSIQUE_OFS
        ld b,MUSIQUE_SPEED
        call dma_play
```

```
; (CPC)
        ld a,MUSIQUE_CH
        ld hl,snd_lists+MUSIQUE_OFS
        ld b,MUSIQUE_SPEED
        call snd_play
```

3.4 bis L'éditeur de musique et l'import de MIDI / MP3

Les musiques (fichiers .snd avec des lignes « A: », « B: », « C: ») s'ouvrent dans un **éditeur en grille** façon tracker : une colonne par voie du PSG, une ligne par pas. Une case contient une note et son volume (en hexadécimal, ou « . » pour le volume de la voie), « === » pour un silence, ou rien : la note précédente continue. L'onglet **Texte** montre le fichier produit ; les bruitages (instructions directes TONE, PAUSE…) s'éditent toujours en texte.



L'éditeur de musique : grille des 3 voies, réglages, piano

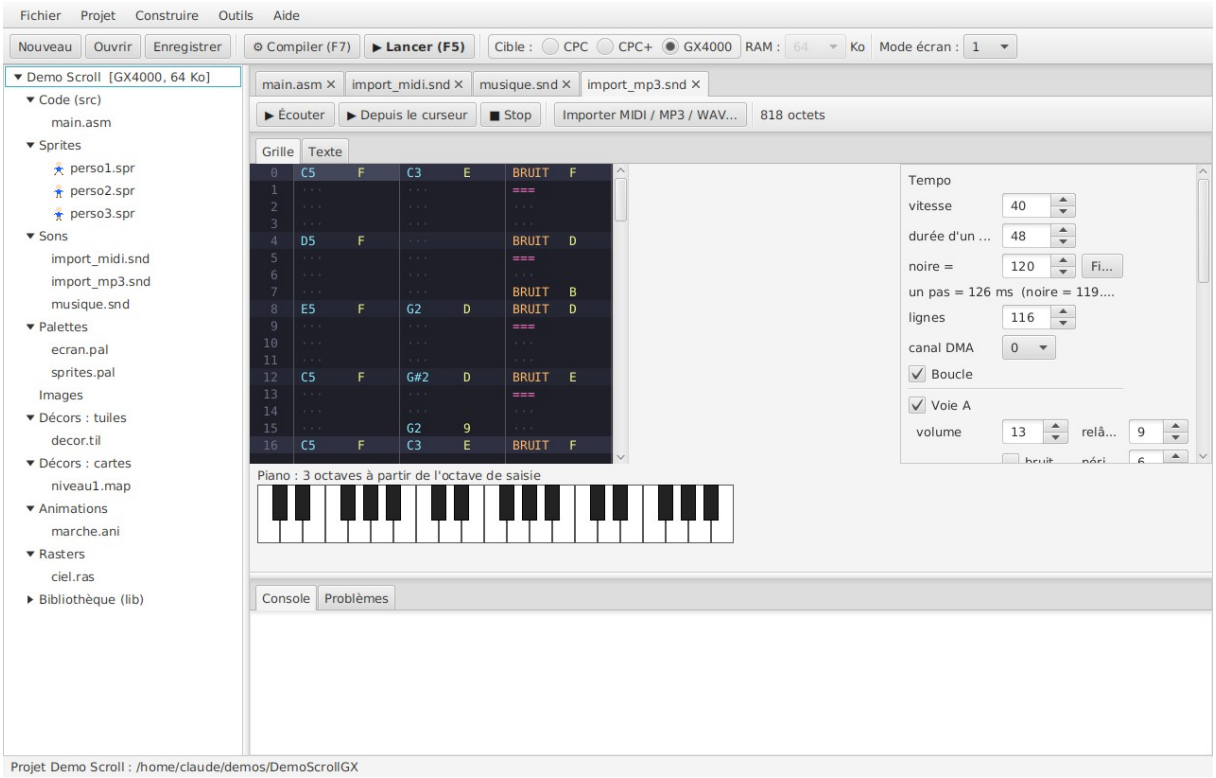
- **Saisie au clavier** (disposition AZERTY, comme les trackers) : W S X D C V G B H N J , = do à si de l'octave de saisie ; A É Z " E R (T - Y È U = l'octave au-dessus. Espace = silence, Suppr = vider, Tab ou ← → = changer de voie, Maj + ↑↓ = sélection, Ctrl+C / Ctrl+V = copier / coller des lignes. Chaque note saisie est jouée.

- **Piano** : un clic sur une touche écrit la note dans la case courante.
- **Réglages** : vitesse et durée d'un pas (ou « noire = 120 » puis Fixer), nombre de lignes, boucle, canal DMA ; par voie : volume, relâche (articulation), bruit (la voie joue du bruit, pour la batterie).
- **Transposer** la sélection (±1 demi-ton, ±1 octave), insérer / supprimer une ligne.
- **Écouter** tout le morceau ou à partir du curseur (la ligne jouée est surlignée). La taille de la liste DMA est affichée : au-delà d'environ 6 Ko, vérifier la place en RAM (DMA_RAM sur CPC+/GX4000).

Importer MIDI / MP3 / WAV… remplit la grille à partir d'un fichier externe :

Format	Méthode	Résultat
MIDI (.mid)	lecture des notes (tempo du fichier) ; la partie la plus aiguë va sur A, la plus grave sur B, les autres sur C ; la batterie (canal 10) peut devenir du bruit sur C	fidèle (notes exactes) : la meilleure solution quand un MIDI du morceau existe
MP3, OGG, FLAC, M4A	conversion en WAV par ffmpeg (sudo apt install ffmpeg), puis comme un WAV	approximation « chiptune »
WAV	pas à pas : spectre (FFT), somme des harmoniques pour trouver la mélodie (A), la basse (B) et une voix intermédiaire (C) ; attaques aiguës -> bruit ; tempo détecté automatiquement si 0	bon pour une mélodie claire ; un morceau très chargé donne un résultat plus confus

Les options : tempo (0 = automatique), pas par temps (4 = double croche), début et durée maximale (une minute de musique fait plusieurs Ko), transposition, batterie en bruit, articulation, sensibilité (audio). Le résultat se retouche ensuite dans la grille. La même musique est jouée par le DMA sur CPC+/GX4000 et par le lecteur logiciel sur CPC. Le compilateur n'écrit un registre du PSG que s'il change, ce qui réduit la taille des listes.



Une mélodie importée depuis un MP3 (tempo détecté, notes retrouvées)

```
./gxstudio.sh import morceau.mid assets/sons/theme.snd          (en ligne de commande)
./gxstudio.sh import morceau.mp3 assets/sons/theme.snd 120 30   (tempo 120, 30 secondes)
```

3.4 ter Jouer un son ou une musique depuis le programme

Important : créer un fichier .snd (à la main, dans la grille ou par import MIDI/MP3) ne suffit pas pour l'entendre dans le jeu. Le studio le **compile**, mais il ne sera joué que si le programme en assembleur **appelle le lecteur**. Un son jamais appelé occupe de la place dans la cartouche et reste muet.

À chaque compilation, tous les fichiers de assets/sons sont convertis dans **build/gen.asm**. Pour un fichier `musique.snd`, le nom en majuscules sert de préfixe à trois équates :

Équate	Contenu	Vient de
MUSIQUE_OFS	position de la liste parmi toutes les listes de son (en octets)	calculé par le studio
MUSIQUE_SPEED	diviseur : 1 tic = SPEED + 1 lignes d'écran	ligne « vitesse » du fichier
MUSIQUE_CH	canal de lecture conseillé (0, 1 ou 2)	ligne « canal » du fichier

Un nom de fichier avec des caractères spéciaux est simplifié : `son-tir.snd` donne SON_TIR_OFS. En cas de doute, ouvrir build/gen.asm (ou F12 sur le nom dans le code).

Sur CPC+ et GX4000, le DMA de l'ASIC lit les listes en **RAM** (pas dans la cartouche) : la bibliothèque les recopie à l'adresse DMA_RAM avec `dma_init`, à appeler **une fois** au démarrage (les projets de départ le font déjà). **Sur CPC**, le programme est déjà en RAM : les listes sont à l'étiquette `snd_lists` et le lecteur logiciel est appelé tout seul par l'interruption, il n'y a rien à initialiser.

```
; --- CPC+ / GX4000 : une fois au demarrage
        call dma_init                   ; copie des listes en RAM (DMA_RAM)

; --- lancer la musique                ; --- meme chose sur CPC
        ld a,MUSIQUE_CH                  ld a,MUSIQUE_CH
        ld hl,DMA_RAM+MUSIQUE_OFS        ld hl,snd_lists+MUSIQUE_OFS
        ld b,MUSIQUE_SPEED               ld b,MUSIQUE_SPEED
        call dma_play                    call snd_play

; --- arreter
        ld a,MUSIQUE_CH                  ld a,MUSIQUE_CH
        call dma_stop                    call snd_stop
```

Routine (CPC+/GX)	Routine (CPC)	Rôle
dma_init	—	recopie les listes en RAM, coupe les canaux (au démarrage)
dma_play	snd_play	A = canal, HL = liste, B = diviseur : (re)lance la liste depuis le début
dma_stop	snd_stop	A = canal : arrête ce canal
dma_stop_all	snd_stop_all	arrête tout et coupe le volume des 3 voies
dma_tempo	snd_tempo	A = nouveau diviseur du canal 0 : accélérer la musique en cours

Les règles à connaître :

- Il y a **3 canaux** qui jouent en même temps. Un `dma_play` sur un canal déjà occupé **coupe** ce qu'il jouait. L'habitude : la musique sur le canal 0, les bruitages sur les canaux 1 et 2 (ligne « canal » du fichier).
- Un canal n'est pas une voie : une liste peut écrire dans n'importe quelle voie du PSG (A, B, C). Si la musique utilise les voies A et B, les bruitages doivent utiliser la voie C, sinon ils se marchent dessus.
- On lance un son **une seule fois**, au moment de l'événement (collision, tir...), jamais à chaque image : un `dma_play` répété 50 fois par seconde relance sans cesse le début et ne produit qu'un bourdonnement.

- Une musique avec la ligne `boucle` tourne seule jusqu'au `dma_stop` ; le jeu n'a plus à s'en occuper.
- On peut écrire sa propre petite routine pour ne pas répéter les trois lignes, comme `snd_rebond` dans le casse-brique.

Mon son ne s'entend pas ? Vérifier dans l'ordre : le nom existe dans build/gen.asm (sinon erreur « keyword not found » à la compilation) ; `dma_init` est appelé au démarrage (CPC+/GX) ; l'appel `dma_play` est bien exécuté (point d'arrêt F9 dessus) ; aucun autre son n'est lancé juste après sur le même canal ; le son n'est pas relancé à chaque image.

Dans les exemples fournis :

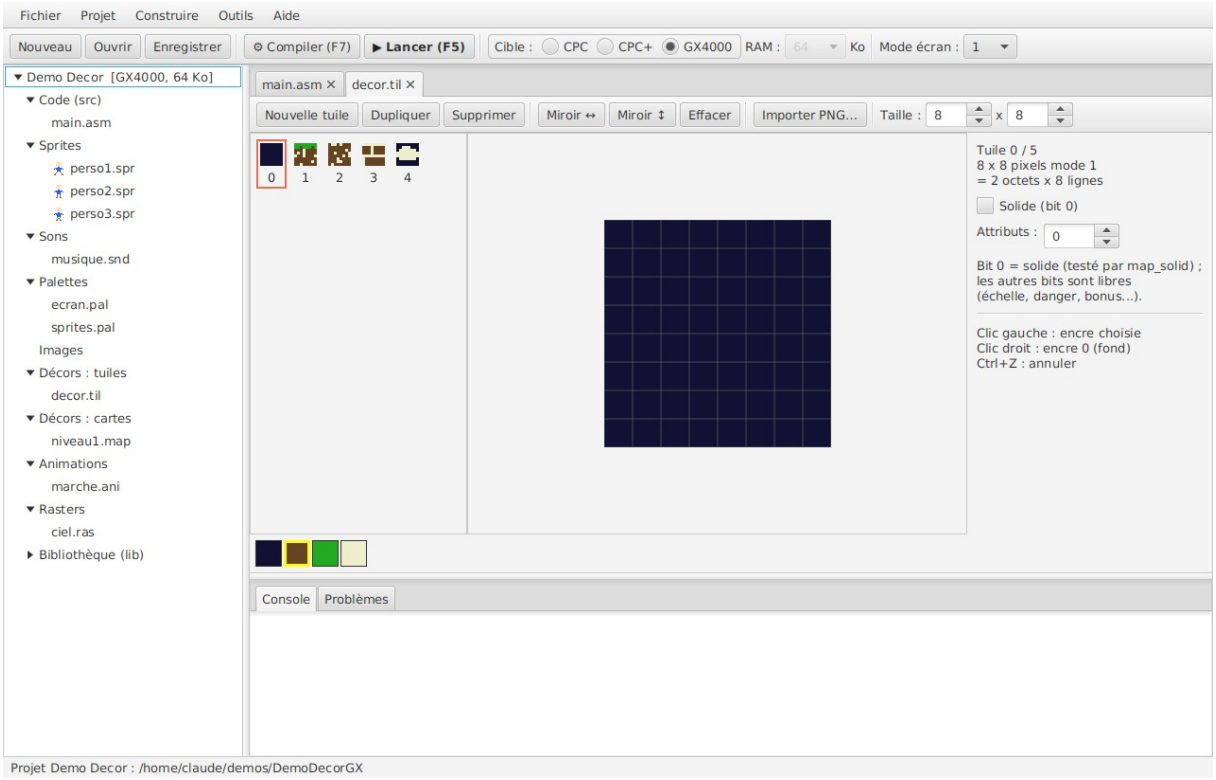
Exemple	Sons du projet	Joués par le programme
CasseBrique	musique, rebond, brique, perdu	tous : la musique sur l'écran titre, les bruitages dans les collisions (snd_rebond...)
DemoScrollGX	musique, import_midi, import_mp3	import_midi seulement ; pour écouter le MP3 converti, remplacer IMPORT_MIDI par IMPORT_MP3 dans les trois lignes de l'appel
DemoScrollCPC	musique, import_midi	import_midi (lecteur logiciel, snd_play)
DemoDecorGX / CPC	musique	aucun : démonstrations d'affichage ; le son est là pour être essayé dans l'éditeur

3.5 Les images

Une image PNG placée dans assets/images (Projet > Importer une image) est convertie dans le mode écran du projet avec l'encre la plus proche : étiquettes `IMG_NOM`, `IMG_NOM_W` (largeur en octets), `IMG_NOM_H` (hauteur). Sur CPC+/GX4000 elle va dans la page 2 de la cartouche ; sur CPC dans le programme. Sur un CPC de 128 Ko ou plus, nommer le fichier **nom@4.png** range l'image dans la banque 4 (5, 6, 7… au-delà) : le studio crée le fichier BANK4.BIN sur la disquette, le programme le charge au démarrage avec le firmware, et `bank_set` (avec `IMG_NOM_BANK`) rend la banque visible en &4000.

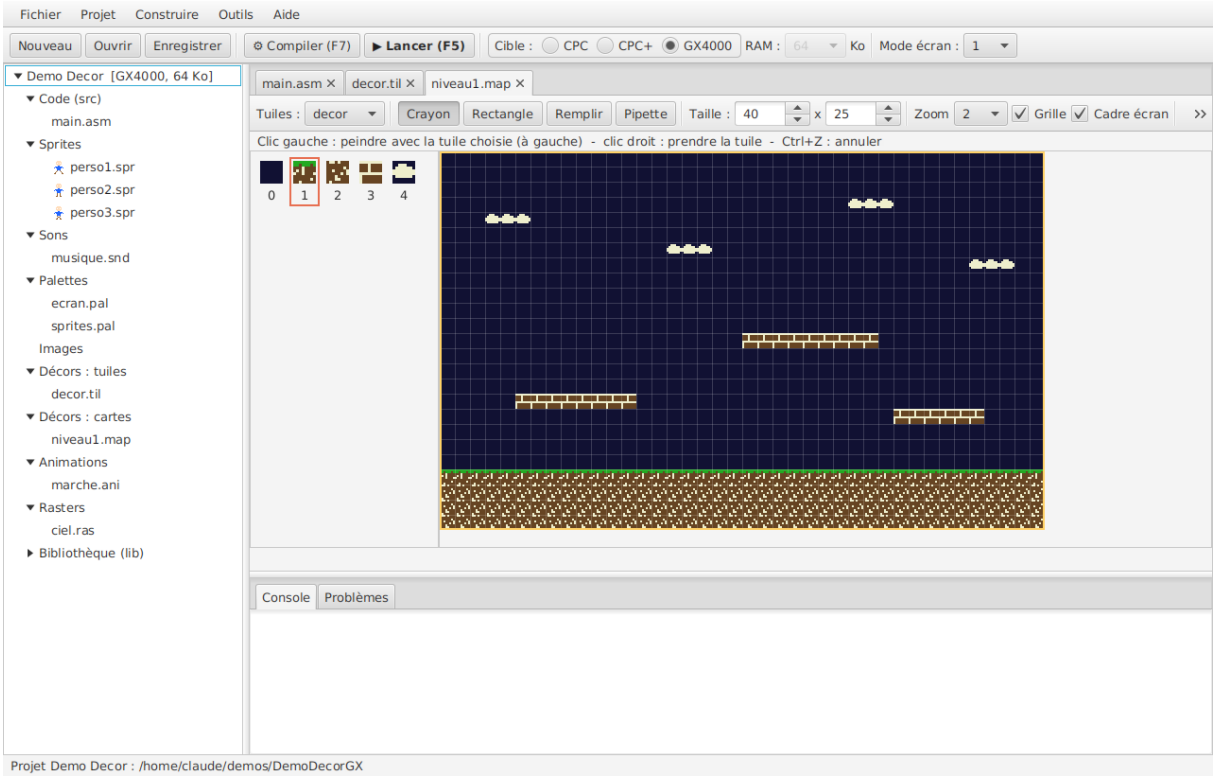
3.6 Les décors : jeux de tuiles et cartes

Un décor se construit en deux temps. Le **jeu de tuiles** (Décors : tuiles, fichier .til) contient de petites images, de 8 x 8 pixels par défaut, dessinées avec les encres de l'écran ; chaque tuile a un octet d'**attributs** dont le bit 0 signifie « solide » (sol, mur), les autres bits restant libres pour le jeu (échelle, danger, bonus…). La **carte** (Décors : cartes, fichier .map) est un tableau de numéros de tuiles, de la taille que l'on veut : un écran, ou un niveau entier plus large que l'écran.



L'éditeur de jeu de tuiles : liste, dessin, attributs

- Jeu de tuiles : nouvelle tuile, dupliquer, supprimer, miroirs, taille (multiple de 4 pixels en mode 1, de 2 en mode 0), **import d'une image PNG** découpée en tuiles (les tuiles identiques ne sont gardées qu'une fois).
- Carte : crayon, rectangle, remplissage, pipette (ou clic droit), zoom, grille, **cadre d'un écran** (40 x 25 tuiles en mode 1, 20 x 25 en mode 0) pour voir ce qui sera visible. **Importer PNG** transforme un dessin complet en tuiles + carte en une seule opération.
- Ctrl+Z annule ; les cartes se mettent à jour quand on modifie une tuile.

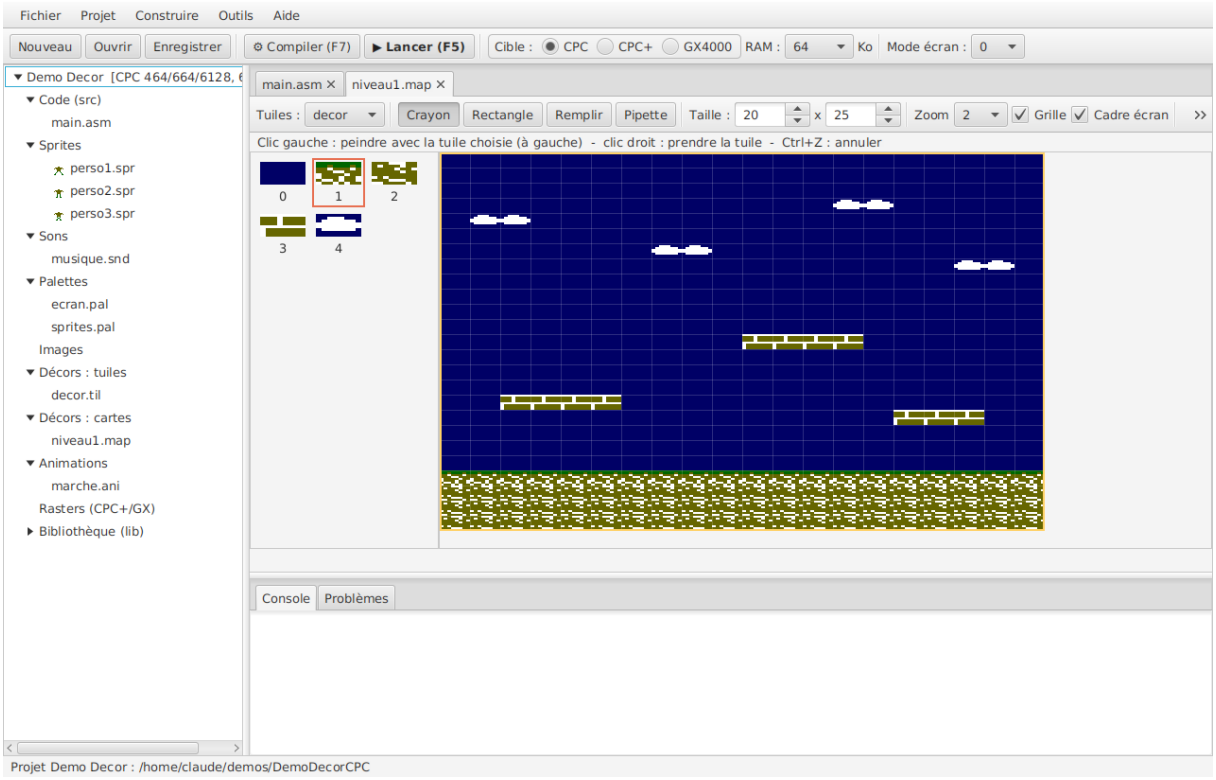


L'éditeur de carte (GX4000, mode 1)

À la compilation, le studio produit pour chaque jeu de tuiles `TILES_NOM` (les graphismes au format de l'écran), `TILES_NOM_W` (largeur en octets), `TILES_NOM_H`, `TILES_NOM_N` et la table d'attributs `TFLAGS_NOM` ; pour chaque carte `MAP_NOM`, `MAP_NOM_W`, `MAP_NOM_H` et `MAP_NOM_TILES`. Les routines de lib/common/tiles.asm (les mêmes sur CPC et CPC+/GX4000) les utilisent :

```
        ld hl,TILES_DECOR                       ; choisir le jeu de tuiles
        ld bc,TILES_DECOR_W*256+TILES_DECOR_H
        call tiles_use
        xor a
        ld (map_x0),a                           ; coin de l'ecran (colonne octet, ligne)
        ld (map_y0),a
        ld hl,MAP_NIVEAU1                       ; dessiner la carte
        ld d,MAP_NIVEAU1_W
        ld bc,MAP_NIVEAU1_W*256+MAP_NIVEAU1_H   ; colonnes, rangees
        call map_draw
        ...
        ld hl,MAP_NIVEAU1                       ; collision : tuile en (B, C) solide ?
        ld d,MAP_NIVEAU1_W
        call map_get                            ; A = numero de tuile
        ld hl,TFLAGS_DECOR
        call map_solid                          ; NZ = solide
```

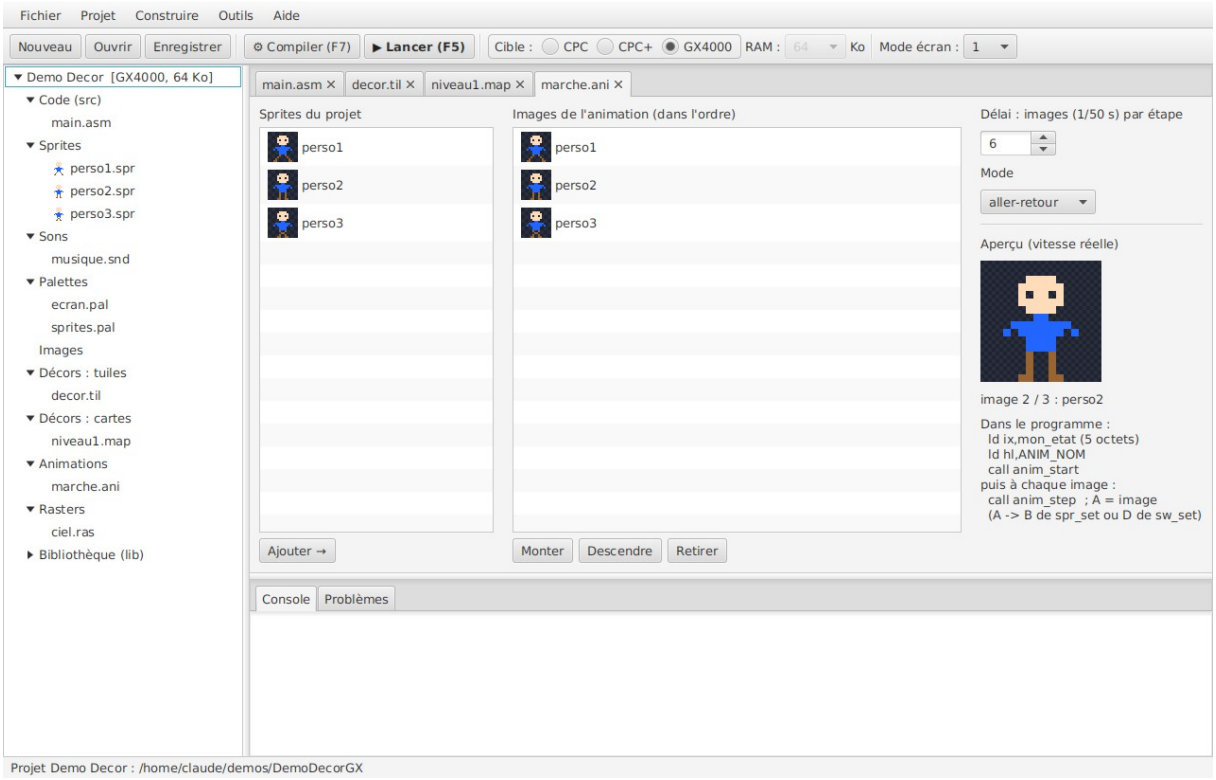
Pour faire défiler un niveau plus large que l'écran, on redessine une fenêtre de la carte : `map_draw` commence à n'importe quelle case (HL = MAP_NOM + y x largeur + x). Les routines demandent une variable `tile_vars` de 12 octets (déjà réservée dans les projets de départ).



La même carte pour un CPC en mode 0 (20 x 25 tuiles, pixels deux fois plus larges)

3.7 Les animations de sprites

Une **animation** (fichier .ani) est une suite de sprites du projet, un délai (en 1/50 de seconde par image) et un mode : **boucle**, **une fois** (s'arrête sur la dernière image) ou **aller-retour**. L'aperçu joue l'animation à la vitesse réelle. Le studio produit `ANIM_NOM`, et lib/common/anim.asm la fait avancer :

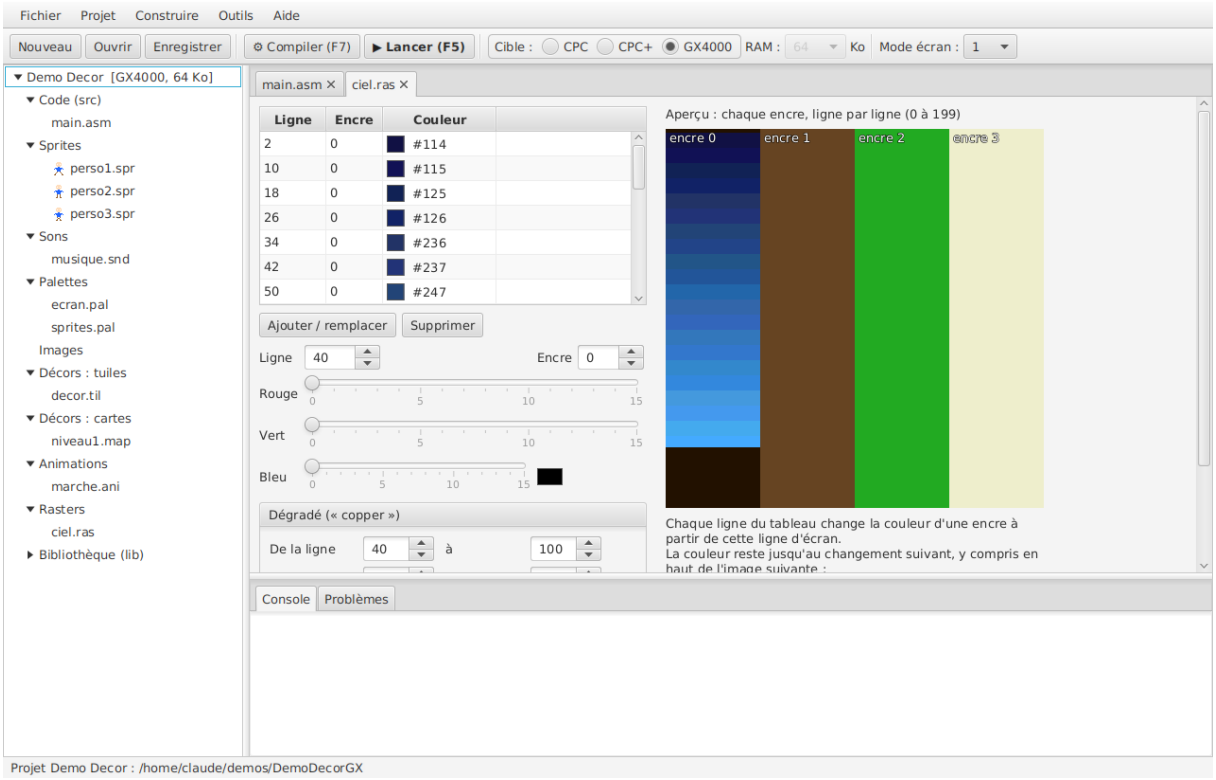


L'éditeur d'animation : sprites disponibles, ordre des images, délai, mode, aperçu

```
        ld ix,etat_perso                    ; 5 octets en RAM
        ld hl,ANIM_MARCHE
        call anim_start
        ...                                 ; a chaque image :
        ld ix,etat_perso
        call anim_step                      ; A = image (F_...), CF = 1 si "une fois"
terminee
        ld b,a                              ; CPC+/GX : B de spr_set   (CPC : D de
sw_set)
```

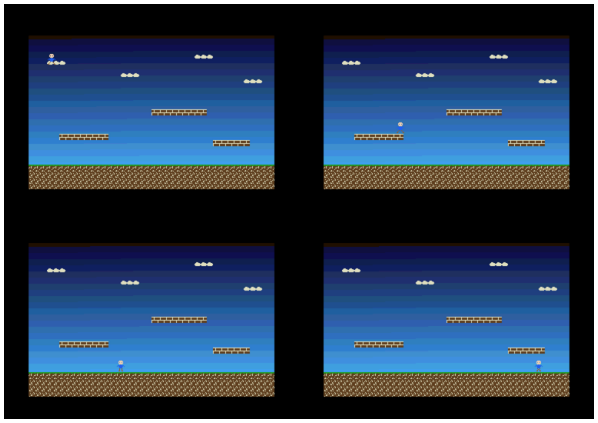
3.8 Les rasters (CPC+ / GX4000)

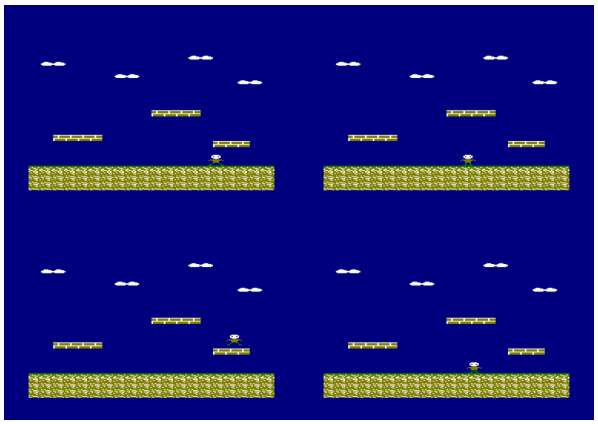
L'éditeur de rasters prépare des **tables de changements de couleur** : à telle ligne, telle encre (0 à 15) prend telle couleur. Le générateur de **dégradé** remplit automatiquement un ciel ou une barre « copper » (de la ligne A à la ligne B, toutes les N lignes, d'une couleur à une autre). L'aperçu montre la couleur de chaque encre ligne par ligne. Les contrôles signalent les lignes trop proches (2 lignes au minimum entre deux changements) ou hors de l'écran.



L'éditeur de rasters : table, réglage d'un changement, dégradé, aperçu

Le studio produit `RASTER_NOM`, une table complète pour la bibliothèque (nouveau **type 4** de lib/plus/raster.asm : n'importe quelle encre), qu'il suffit de lancer : `ld hl,RASTER_CIEL : call raster_start`. Au démarrage, `pri_detect` mesure si la machine compare 8 ou 9 bits du numéro de ligne dans le registre PRI (les émulateurs et la console ne sont pas tous d'accord) et choisit la ligne d'armement de l'image suivante : les tables tournent à 50 images par seconde partout (variable `pri_arm`). La couleur reste en place jusqu'au changement suivant, y compris en haut de l'image suivante. Sur un CPC classique, les tables sont ignorées (les rasters y demandent une synchronisation au cycle près).



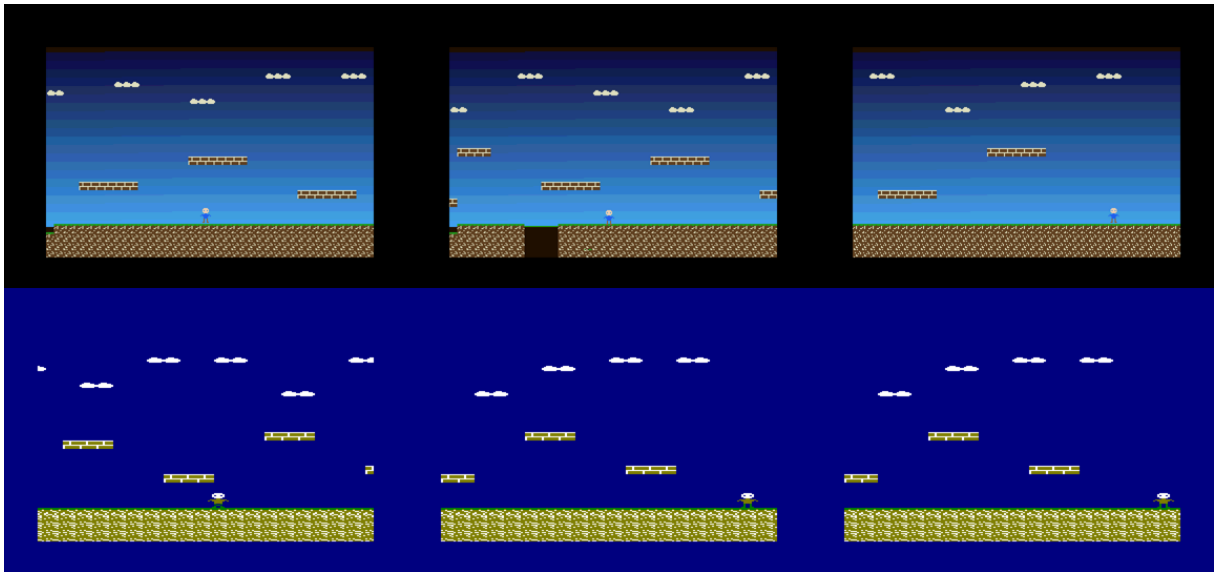


3.9 Le défilement des cartes plus larges que l'écran

lib/common/scroll.asm fait défiler horizontalement une carte de n'importe quelle largeur sans jamais redessiner tout l'écran : on déplace le début de l'écran du CRTC (registres 12 et 13) d'un caractère (2 octets, soit 8 pixels en mode 1 ou 4 en mode 0) et on dessine seulement la colonne de tuiles qui apparaît. Sur **CPC+ / GX4000**, le soft-scroll de l'ASIC décale en plus l'image de 0 à 15 pixels : le défilement est **fluide, au pixel près** (le bord gauche est élargi d'un caractère pour cacher la colonne en cours de dessin). Sur **CPC**, il avance par pas d'un caractère.

```
        ld hl,TILES_DECOR                       ; jeu de tuiles (tuiles de 8 pixels de
large)
        ld bc,TILES_DECOR_W*256+TILES_DECOR_H
        call tiles_use
        ld hl,MAP_NIVEAU1                       ; carte, largeur, rangees affichees
        ld d,MAP_NIVEAU1_W
        ld e,MAP_NIVEAU1_H
        call scroll_init                        ; dessine le debut du niveau
boucle: call wait_frame
        call scroll_apply                       ; debut d'ecran (et decalage fin sur CPC+)
        ...
        ld hl,(camera)                          ; position de la camera (pixels mode 2)
        call scroll_to                          ; avance d'une colonne au plus
        ; sprite materiel : X ecran = X dans le monde - camera
```

- Coordonnées : la caméra et les objets du monde sont en pixels « mode 2 » (16 par caractère). Sur CPC, la caméra réelle est `scr_char` x 16.
- Pendant un défilement, la mémoire écran « tourne » : pour dessiner à l'écran, utiliser `scr_xy` (adresse d'un octet), pas line_addr. Les sprites logiciels du CPC (swsprites.asm) en tiennent compte d'eux-mêmes ; sur CPC, effacer les sprites (sw_restore) avant scroll_apply.
- Les collisions se testent dans la carte avec map_get / map_solid, en coordonnées du monde.
- Projets exemples/DemoScrollGX (fluide, manette, musique importée d'un MIDI) et DemoScrollCPC (même jeu sur CPC 64 Ko).



Défilement d'un niveau trois fois plus large que l'écran : GX4000 (en haut), CPC (en bas)

3.10 Le défilement dans les 8 directions

lib/common/scroll8.asm fait défiler une carte plus grande que l'écran **dans les deux sens** (haut, bas, gauche, droite et diagonales). Il remplace scroll.asm (mêmes noms `scr_off`, `scr_off2`, `scr_xy` : les sprites logiciels du CPC suivent le défilement) ; il demande `VAR scroll_vars,40`.

Le principe reste le même : on déplace le début de l'écran du CRTC et on ne dessine que ce qui

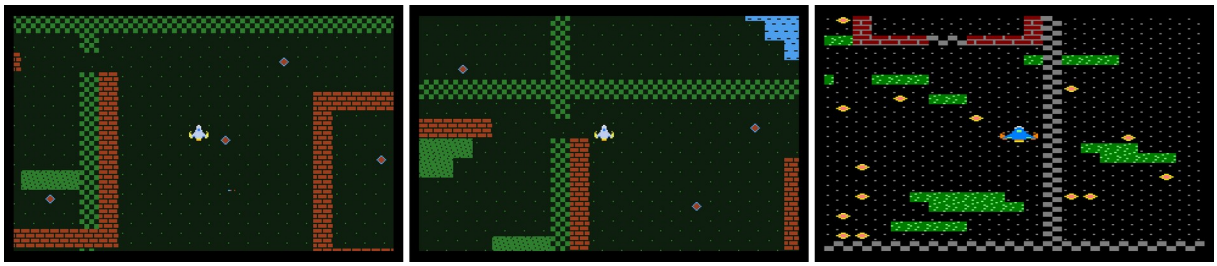
apparaît. L'adresse avance d'**un** pour un caractère vers la droite et de **40** pour une rangée (8 lignes) vers le bas ; chaque bloc de 2 Ko de la mémoire écran boucle sur lui-même. Il faut une rangée de réserve, cachée, pour le décor qui va apparaître : l'écran passe donc à **24 rangées visibles** (192 lignes, registre 6 du CRTC), plus une 25e cachée : 25 x 40 = 1000 caractères, le bloc de 2 Ko en contient 1024.

	CPC+ / GX4000	CPC
Horizontal	au pixel près (soft-scroll de l'ASIC, 0 à 15 pixels mode 2)	par caractère (8 pixels mode 1, 4 pixels mode 0)
Vertical	à la ligne près (soft-scroll vertical de l'ASIC, 0 à 7 lignes)	par rangée de 8 lignes
Écran	40 caractères x 24 rangées	40 caractères x 24 rangées

```
        ld hl,TILES_MONDE                       ; tuiles de 8 ou 16 lignes,
        ld bc,TILES_MONDE_W*256+TILES_MONDE_H   ; 1, 2 ou 4 caracteres de large
        call tiles_use
        ld hl,MAP_MONDE                         ; carte, largeur, hauteur (cases)
        ld d,MAP_MONDE_W
        ld e,MAP_MONDE_H
        call scroll8_init                       ; ecran de 24 rangees
        ld hl,(cam_x)                           ; camera de depart (pixels mode 2, lignes)
        ld de,(cam_y)
        call scroll8_jump                       ; place la camera et dessine l'ecran
boucle: call wait_frame
        call scroll8_apply                      ; debut d'ecran + decalages fins (CPC+)
        ...
        ld hl,(cam_x)
        ld de,(cam_y)
        call scroll8_to                         ; une colonne et une rangee au plus
        ; sprite : X ecran = X monde - (s8_camx), Y ecran = Y monde - (s8_camy)
```

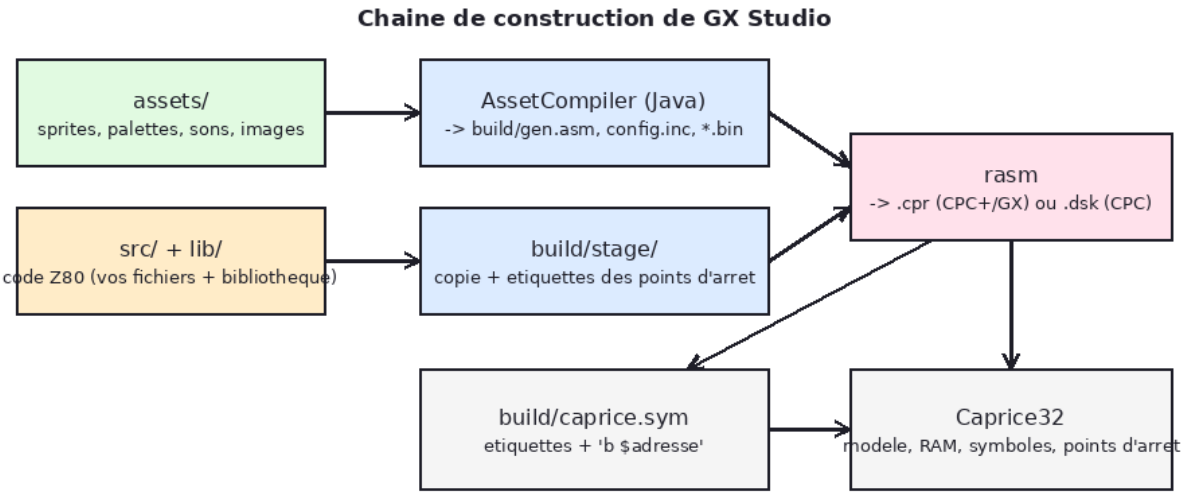
Routine / variable	Rôle
scroll8_init	HL = carte, D = largeur, E = hauteur (cases) : règle l'écran à 24 rangées, dessine le coin haut gauche
scroll8_jump	HL = caméra X (pixels mode 2), DE = caméra Y (lignes) : place la caméra d'un coup (début de niveau, téléportation) et redessine
scroll8_to	même chose en suivant : avance d'une colonne et d'une rangée au plus par image (camera bornée à la carte)
scroll8_apply	envoie le début d'écran au CRTC et les décalages fins à l'ASIC ; juste après wait_frame
s8_camx, s8_camy	caméra réellement affichée : à retrancher des positions du monde pour placer les sprites (sur CPC elle avance par caractère et par rangée)
scr_xy	adresse écran d'un octet (colonne B, ligne C) en tenant compte du défilement

- Tailles de tuiles possibles : 1, 2 ou 4 caractères de large (mode 1 : 8, 16, 32 pixels ; mode 0 : 4, 8, 16 pixels) et 8 ou 16 lignes. Carte : jusqu'à 255 rangées de caractères (2040 lignes) et 255 cases de large.
- Dessin d'une rangée (40 caractères) ou d'une colonne (25 caractères) : une petite partie de l'image ; une diagonale dessine les deux dans la même image.
- Sur CPC, comme pour scroll.asm : `sw_restore_all` (efface les sprites logiciels) avant scroll8_apply, puis `sw_update`.
- Projets exemples/DemoScroll8GX (monde de 48 x 32 tuiles de 16 x 16, fluide, vaisseau en sprite matériel, murs et eau infranchissables) et DemoScroll8CPC (même monde en mode 0 avec un sprite logiciel).

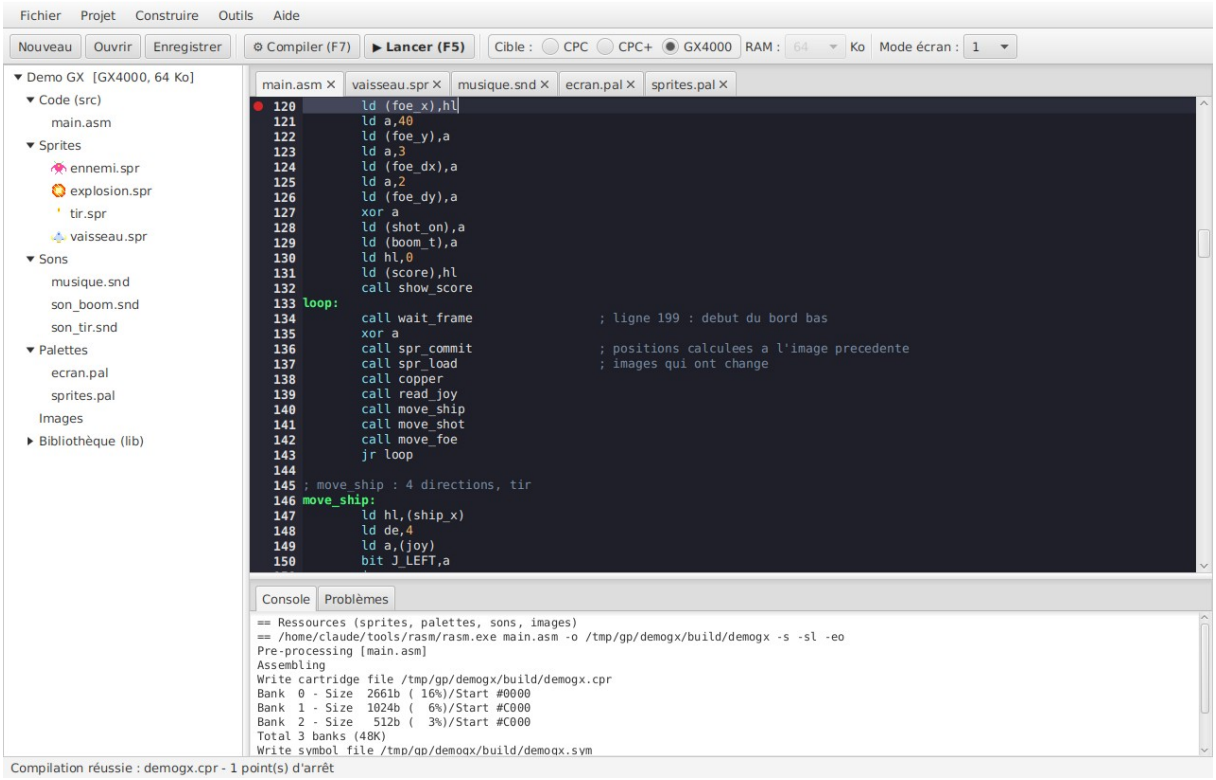


Défilement 8 directions : GX4000 (deux positions, fluide) et CPC (à droite, par caractère et par rangée)

4. Compiler, lancer, déboguer



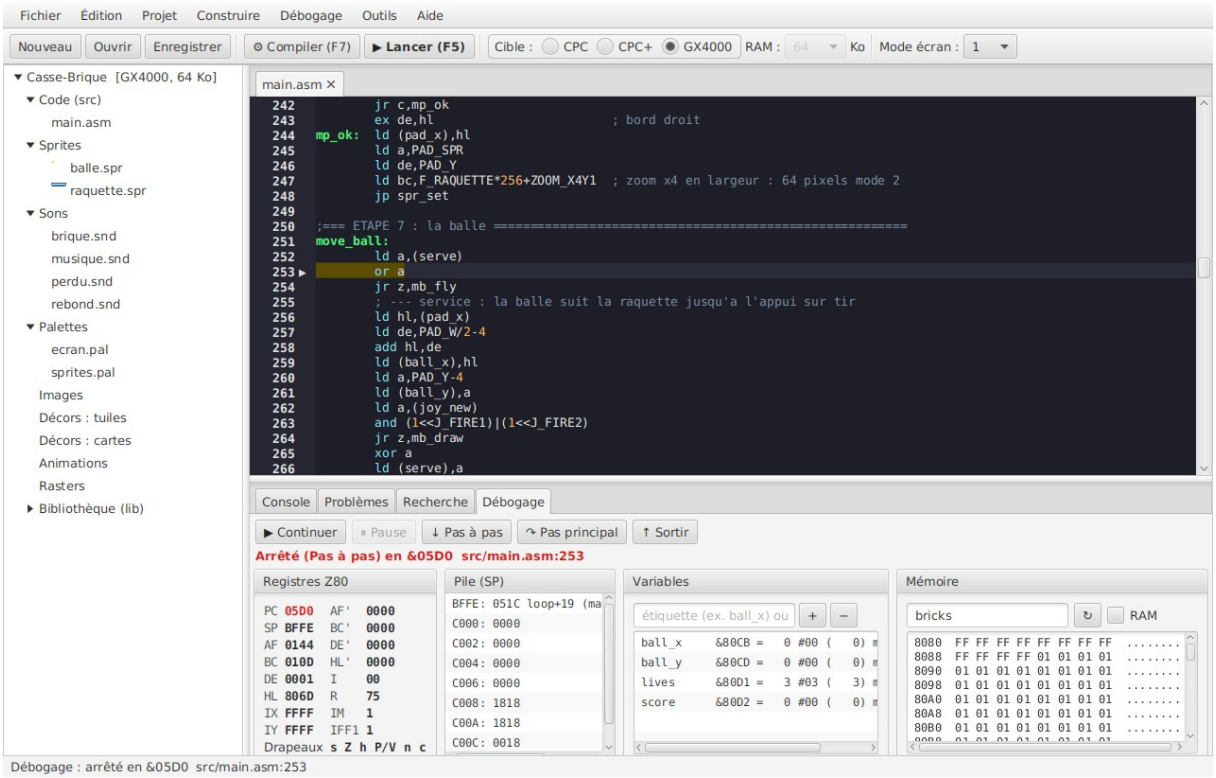
- Les ressources sont converties dans build/ (classe AssetCompiler, l'équivalent Java du script gfx.py de STELLAR WINGS).
- Les sources sont copiées dans build/stage/ ; devant chaque ligne de code, le studio ajoute une étiquette locale `.__lF_L` **sur la même ligne** : les numéros de ligne des erreurs restent justes, votre code n'est jamais modifié, et rasm donne l'adresse de chaque ligne (points d'arrêt, pas à pas, ligne courante). Les lignes des corps de macro et des blocs REPEAT ne sont pas marquées.
- rasm produit la cartouche ou la disquette et la liste des symboles. Les définitions du projet (Projet > Propriétés, par exemple `GOD_MODE=1, START_WAVE=4`) sont passées avec -D.
- Le studio écrit build/caprice.sym : toutes vos étiquettes (`al $adresse .nom`) et les points d'arrêt (`b $adresse`).
- Caprice32 est lancé depuis son dossier avec le bon modèle (664 pour un CPC 64 Ko, 6128 au-delà, 6128+ pour CPC+/GX4000), la bonne mémoire, le jeu (cartouche, ou disquette avec RUN"NOM tapé automatiquement), les symboles et le serveur de débogage.



Un point d'arrêt (rond rouge) et le résultat de la compilation

4.1 Déboguer dans le studio

Avec le Caprice32 corrigé, **F5** relie le studio à l'émulateur (la console affiche « Débogueur relié à Caprice32 »). Le programme tourne normalement ; dès qu'il s'arrête (point d'arrêt, pause, pas à pas), le studio ouvre le fichier concerné, **surligne la ligne** où il est arrêté (flèche jaune dans la marge) et remplit l'onglet **Débogage**. Cela marche aussi dans la bibliothèque (lib/) : un pas à pas dans un CALL ouvre la routine.



Arrêt sur un point d'arrêt puis pas à pas : ligne courante, registres (en rouge ceux qui ont changé), pile, variables, mémoire

Touche (menu Débogage)	Action
F5	compiler et lancer avec le débogueur
F9 ou clic dans la marge	poser / enlever un point d'arrêt, **même pendant l'exécution** (envoyé aussitôt à Caprice32)
F6	pause : arrête le programme là où il est
F8	continuer
F11	pas à pas : une instruction (entre dans les CALL)
F10	pas principal : une instruction ; un CALL, RST, LDIR, DJNZ… est exécuté en entier
Maj+F11	sortir : continue jusqu'au RET de la routine en cours
Ctrl+Maj+W	ajouter aux variables l'étiquette sous le curseur
Maj+F5	arrêter l'émulateur

- **Registres** : PC, SP, AF, BC, DE, HL, IX, IY, registres secondaires, I, R, mode d'interruption, drapeaux (S Z H P/V N C, majuscule = 1). Une valeur qui a changé depuis l'arrêt précédent est en rouge ; une infobulle donne l'étiquette correspondante.
- **Pile** : 8 mots à partir de SP ; une adresse de retour est traduite en « routine+n (fichier:ligne) ».
- **Variables** : étiquettes ou adresses surveillées (ex. ball_x, lives, &8000), avec l'octet en décimal, hexadécimal et signé, et le mot de 16 bits. La liste est gardée dans le fichier .watches du projet.
- **Mémoire** : 256 octets à partir d'une adresse ou d'une étiquette (`bricks`, `spr_tab+5`). Sur CPC+/GX4000 la cartouche masque &0000-&3FFF et &C000-&FFFF en lecture : cocher **RAM** pour voir l'écran ou ce qui est dessous.
- Si le source a été modifié depuis la compilation, les adresses des nouveaux points d'arrêt sont approximatives : recompiler (F5).
- Avec un Caprice32 sans le correctif, la console l'indique et le débogage se fait comme avant dans les outils de Caprice32 (Maj+F2), qui s'ouvrent au point d'arrêt avec vos étiquettes.

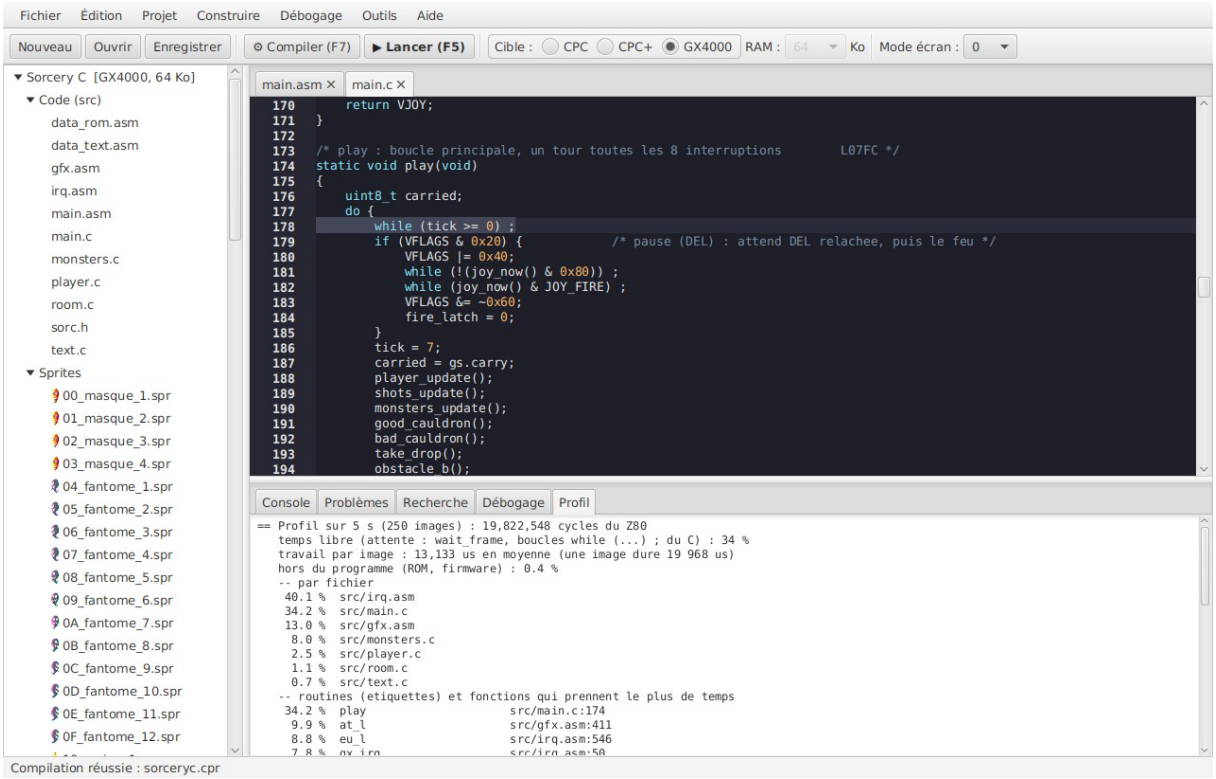
Le serveur ajouté à Caprice32 est un petit protocole texte sur 127.0.0.1 (option `-O devtools.server_port=6543`) : une commande par ligne (pause, cont, step, over, out, regs, mem, ram, poke, bp add/del/clear, shot, key), utilisable aussi depuis un script, par exemple pour des tests automatiques en TP.

Les mêmes opérations existent en ligne de commande, pratique pour des scripts ou des TP :

```
./gxstudio.sh new   ~/jeux/demo "Demo" GX4000 64
./gxstudio.sh build ~/jeux/demo src/main.asm:120     (point d'arret ligne 120)
./gxstudio.sh run   ~/jeux/demo
```

4.2 Mesurer les performances (profil)

Débogage > Mesurer les performances (Ctrl+F6), ou le bouton ⏱️ **Profil 5 s** de l'onglet Débogage : pendant 5 secondes, le Caprice32 corrigé compte les cycles du Z80 de chaque instruction exécutée ; le studio les regroupe et affiche le rapport dans l'onglet **Profil** (et dans la console).



Le profil de Sorcery C : le temps libre (la boucle d'attente de play), les interruptions, l'affichage, puis les fonctions C

- **Temps libre** : le temps passé à attendre — dans gx_wait_frame / wait_frame (bibliothèque), et, depuis la 0.5.1, dans les boucles d'attente du C de la forme `while (condition) ;` (corps vide), comme `while (tick >= 0) ;`.
- **Travail par image** : le temps restant, en microsecondes par image de 19 968 µs. S'il approche ou dépasse une image, le jeu ralentit.
- **Par fichier, par routine, par ligne** : les fonctions C sont nommées comme dans le source (hit_check, play…) ; les routines de la bibliothèque de SDCC (multiplications, divisions) sont signalées.
- **Pages de cartouche aux mêmes adresses** : quand une page est recopiée en RAM à une adresse déjà occupée par une autre page (par exemple la page 0 de démarrage et le programme recopié en &0000, comme dans Sorcery C), le profil, la pile et le débogueur prennent les noms et les lignes de la page de numéro le plus grand.

Le serveur de débogage accepte aussi `prof on`, `prof off`, `prof clear` et `prof save fichier` (65 536 compteurs de 32 bits, un par adresse) : on peut faire ses propres analyses (Stellar Wings C et Sorcery C en donnent des exemples).

5. Les bibliothèques Z80

Les modèles (dossier templates/) contiennent les bibliothèques copiées dans chaque projet. **Aide > Routines de la bibliothèque** (F1) affiche la liste des routines du projet ouvert avec leurs paramètres.

L'écran de présentation



L'écran de présentation affiché au démarrage de chaque jeu

Depuis la version 0.5.1, tout jeu construit avec la bibliothèque du studio (asm ou C, CPC, CPC+ ou GX4000) affiche au démarrage, pendant **3 secondes**, le logo FANOU 1972 et le texte « Jeux développé avec GXStudio By Fanou ». C'est le démarrage de la bibliothèque (lib/plus/boot.asm, lib/cpc/boot_cpc.asm) qui appelle `gx_splash` (lib/common/splash.asm), avant le `main` du jeu : rien à écrire dans le programme. L'écran est en mode 0 (16 encres) ; à la fin, la palette est remise au noir, l'écran effacé et le mode du jeu rétabli.

Les données (lib/common/splash_data.asm : logo compacté, petite police 8 x 16 des caractères utilisés, couleurs) sont produites par **tools/mksplash.py** et prennent environ 2,6 Ko dans le programme (page 0 d'une cartouche). Pour un projet existant, recopier lib/plus/boot.asm (ou lib/cpc/boot_cpc.asm) et lib/common/splash*.asm depuis templates/lib.

Fichier	Machines	Contenu
lib/plus/boot.asm	CPC+, GX4000	démarrage de la cartouche, déverrouillage de l'ASIC, palettes, PSG
lib/plus/raster.asm	CPC+, GX4000	interruptions raster chaînées : couleurs (copper), bandes de sprites (multiplexage)
lib/plus/sprites.asm	CPC+, GX4000	ombre des 16 sprites matériels, chargement des images
lib/plus/dma.asm	CPC+, GX4000	son DMA : dma_play, dma_stop, dma_tempo
lib/plus/text1.asm	CPC+, GX4000	texte et images en mode 1
lib/cpc/boot_cpc.asm	CPC	démarrage depuis la disquette, chargement des banques, arrêt du firmware, interruptions, palette (27 couleurs), bank_set
lib/cpc/swsprites.asm	CPC	sprites logiciels masqués avec sauvegarde du décor : sw_set, sw_hide, sw_update, sw_restore_all
lib/common/sound_sw.asm	CPC	lecteur logiciel des listes DMA : snd_play, snd_stop, snd_tempo
lib/common/input.asm	toutes	manettes 1 et 2 (+ flèches, Espace, COPY si HAS_KEYBOARD)
lib/common/text0.asm	toutes	texte en mode 0 (20 colonnes)
lib/common/tiles.asm	toutes	décors : tiles_use, tile_draw, map_draw, map_get, map_solid
lib/common/anim.asm	toutes	animations de sprites : anim_start, anim_step
lib/common/scroll.asm	toutes	défilement horizontal : scroll_init, scroll_to, scroll_apply, scr_xy

lib/common/scroll8.asm	toutes	défilement 8 directions : scroll8_init, scroll8_jump, scroll8_to, scroll8_apply, scr_xy

Les deux familles suivent le même schéma : une boucle `wait_frame` → affichage des sprites (`spr_commit` / `spr_load` sur CPC+, `sw_update` sur CPC) → lecture des manettes → logique du jeu. Sur CPC, `wait_frame` attend la synchronisation verticale ; sur CPC+, la fin de la zone visible (ligne 199).

```
sw_draw:
        ld l,(ix+3)
        ld h,(ix+4)
        ld a,(hl)
        ld (ix+8),a                     ; largeur
        inc hl
        ld a,(hl)
        ld (ix+9),a                     ; hauteur
        inc hl
        push hl
        ld b,(ix+1)
        ld c,(ix+2)
        call sw_xy                      ; HL = adresse ecran (defilement compris)
        ld (ix+6),l
        ld (ix+7),h
        ld (ix+5),1
        ex de,hl                        ; DE = ecran
        exx
        ld l,(ix+10)
        ld h,(ix+11)                    ; HL' = tampon
        exx
        pop hl                          ; HL = masques / donnees
        ld c,(ix+9)
swd_r:  push de
        ld b,(ix+8)
swd_c:  ld a,(de)
        exx
        ld (hl),a
        inc hl
        exx
        and (hl)
        inc hl
        or (hl)
        inc hl
        ld (de),a
        call sw_inc
        djnz swd_c
        pop de
        call sw_down
        dec c
        jr nz,swd_r
        ret

; sw_inc : DE = DE + 1 dans la memoire ecran circulaire (bloc de 2 Ko)
sw_inc:
        inc e
```

Le lecteur de son du CPC décompte les lignes comme le DMA : chaque interruption retire 52 lignes au délai en cours, puis exécute les instructions jusqu'à la pause suivante.

```
snd_exec:
        ld l,(ix+1)
        ld h,(ix+2)
        ld b,64                         ; securite : 64 instructions au plus
se_l:   ld e,(hl)
        inc hl
        ld d,(hl)
        inc hl
        ld a,d
        and #F0
        jr z,se_load
        cp #10
        jr z,se_pause
        cp #20
```

```
        jr z,se_rep
        bit 0,e
        jr nz,se_loop
```

6. Programmer en C

Depuis la version 0.5, un jeu peut être écrit en **C**, compilé pour le Z80 par **SDCC** (Small Device C Compiler), et mélangé librement avec de l'assembleur. Le C sert à la logique du jeu (déplacements, collisions, score, niveaux) ; l'affichage, les sprites, le son et les rasters restent assurés par la bibliothèque Z80 du studio, la même que pour les projets en assembleur, appelée depuis le C par de petites fonctions **gx_…**. Les routines qui doivent être très rapides peuvent toujours être écrites en assembleur et appelées depuis le C.

	Projet en assembleur	Projet en C
Programme	src/main.asm	src/main.c (et d'autres .c), src/main.asm assemble la cartouche
Bibliothèque	lib/plus ou lib/cpc, lib/common	les mêmes + lib/c (gx.h, gx_c.asm, gx.c)
Ressources	build/gen.asm (étiquettes, équates)	les mêmes + build/gen.h et build/config.h pour le C
Compilation	rasm	SDCC pour chaque .c, traduction pour rasm, puis rasm (une seule passe pour tout)
Débogage	points d'arrêt, pas à pas sur les lignes .asm	les mêmes sur les lignes .c (et .asm)
Taille du code	référence	environ 1,5 fois plus gros (casse-brique : 2,9 Ko en assembleur, 4,3 Ko en C)

6.1 Installer SDCC

```
sudo apt install sdcc          (Debian / Ubuntu : SDCC 4.x)
sdcc --version                 vérification
```

Par défaut le studio appelle `sdcc` (celui du PATH). Pour une autre version, indiquer son chemin dans **Outils > Réglages**, champ **sdcc**. Les fonctions de la bibliothèque de SDCC (multiplication, division, décalages 16/32 bits, memcpy…) sont prises dans le dossier d'installation de SDCC (/usr/share/sdcc/lib/src) **seulement si le programme les utilise**.

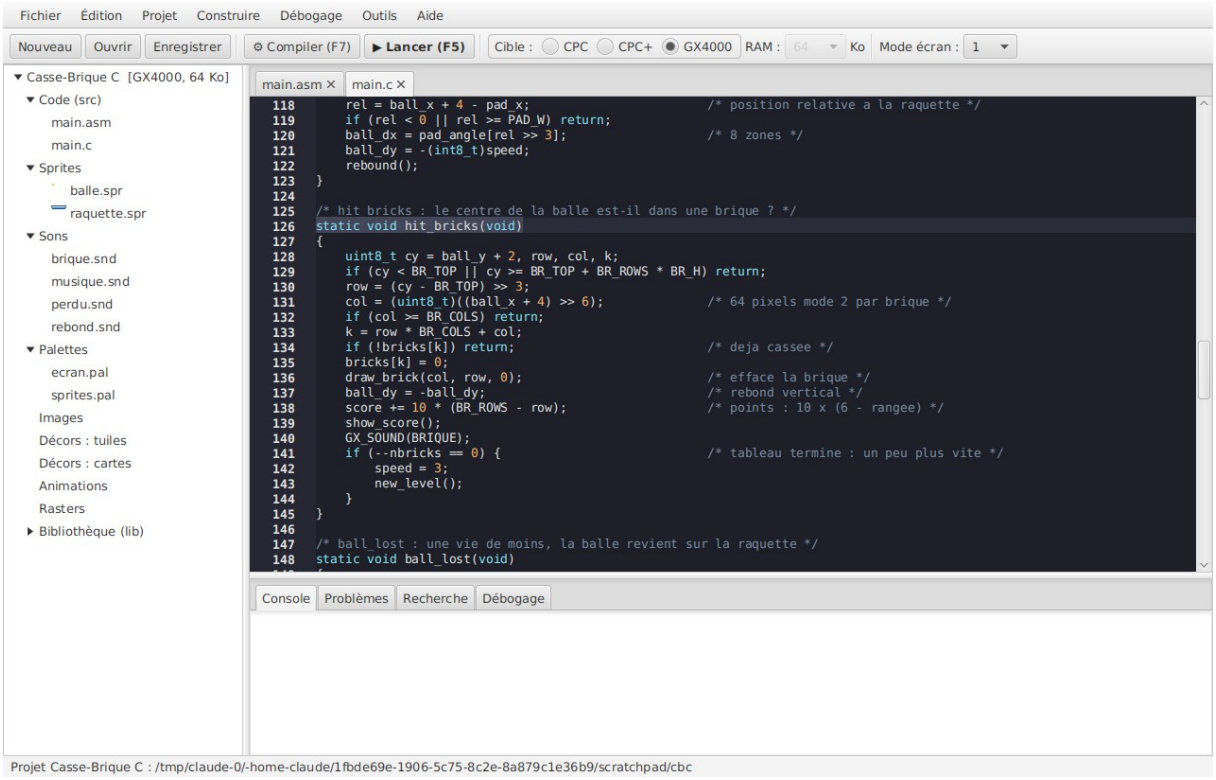
6.2 Créer un projet en C

Fichier > Nouveau projet, langage **C (SDCC) + assembleur**. Le studio crée le même projet de départ qu'en assembleur (un vaisseau qui tire sur un ennemi, musique, bruitages), mais écrit en C. Il se lance tout de suite avec F5. En ligne de commande : `./gxstudio.sh new DOSSIER "Nom" GX4000 64 1 C`.



Nouveau projet : choix du langage

Fichier	Rôle
src/main.c	le jeu : une fonction main() appelée en boucle par main.asm
src/main.asm	la cartouche (ou la disquette) : démarrage, variables de la bibliothèque, pages, inclusion de la bibliothèque, de la liaison C et du code C traduit ; on y ajoute ses routines en assembleur
lib/c/gx.h	l'API du C : prototypes des fonctions gx_…, masques de la manette, zooms, macros
lib/c/gx_c.asm	la liaison : chaque fonction gx_… lit ses paramètres sur la pile et appelle la routine de la bibliothèque
lib/c/gx.c	les fonctions de la bibliothèque écrites en C (gx_text_num, gx_rand)
build/gen.h, build/config.h	générés : les ressources et la configuration du projet vues du C
build/c/*.s, build/c_code.asm, build/c_vars.inc	générés : sortie de SDCC, code traduit pour rasm, taille des variables du C



Un fichier C dans l'éditeur : coloration du C, point d'arrêt avec F9

6.3 Comment le C est compilé

- Le studio convertit d'abord les ressources (comme pour un projet en assembleur) et écrit **build/gen.h**.
- Chaque fichier .c de src/ et de lib/c est compilé par `sdcc -mz80 --reserve-regs-iy --opt-code-size -S` ; les définitions de **Projet > Propriétés** sont passées en -D (comme à rasm).
- La sortie de SDCC (syntaxe sdas) est traduite pour rasm dans **build/c_code.asm** : le code et les constantes vont dans la page 0 avec l'assembleur, les variables en RAM, dans le bloc **c_ram** réservé par main.asm. Les noms `static` d'un fichier sont préfixés pour ne pas se mélanger avec ceux des autres fichiers.
- rasm assemble le tout en une seule passe : le C et l'assembleur se voient mutuellement, sans éditeur de liens. Une étiquette manquante est signalée avant même l'appel de rasm.

Au démarrage, `c_init` (appelé par main.asm) recopie les valeurs initiales des variables globales

(`static uint8_t n = 5;`) et met les autres à zéro, comme le veut le C.

6.4 Écrire du C pour le Z80

SDCC est un vrai compilateur C (C99/C11), mais le Z80 est un processeur 8 bits à 4 MHz : quelques habitudes donnent un code bien plus petit et rapide.

- Utiliser `uint8_t` / `int8_t` (stdint.h, inclus par gx.h) chaque fois que c'est possible ; `int` fait 16 bits, `long` 32 bits (coûteux).
- Préférer des variables **globales static** aux grosses variables locales : les locales sont lues par IX, plus lentement.
- Les tableaux `const` restent dans la cartouche (ROM) ; les autres sont en RAM.
- Multiplier ou diviser par une puissance de 2 avec des décalages (`x >> 3`) ; une vraie division appelle une routine de SDCC.
- Pas de printf, malloc ni fichiers : il n'y a pas de système. Pour le texte : gx_text et gx_text_num.
- Le code du jeu et la bibliothèque doivent tenir dans la page 0 (16 Ko) : l'`assert code_end<=#4000` de main.asm prévient si ce n'est pas le cas.

6.5 L'API gx.h

Toutes les fonctions sont déclarées avec `__sdcccall(0)` (macro GXAPI) : paramètres sur la pile, résultat dans L ou HL. Les coordonnées et numéros sont ceux de la bibliothèque Z80 (chapitre 5).

Fonction	Machines	Rôle
gx_init()	toutes	à appeler au début de main() : son, sprites
gx_wait_frame(), gx_wait_frames(n)	toutes	attendre la fin de l'image (50 par seconde)
gx_read_joy() ; joy, joy_new	toutes	manettes : joy = état, joy_new = boutons qui viennent d'être appuyés (JOY_LEFT, JOY_FIRE…)
gx_clear_screen(), gx_palette(pal_screen), gx_sprite_palette(pal_sprites)	toutes	écran et couleurs
gx_ink(n), gx_text(col, ligne, "…"), gx_text_num(col, ligne, valeur, chiffres)	toutes	texte (col en caractères, ligne en lignes de pixels)
GX_LINE(y)	toutes	adresse écran de la ligne y (table line_addr) pour dessiner soi-même
GX_SOUND(NOM), gx_sound_stop(canal), gx_sound_stop_all(), gx_psg(reg, val)	toutes	sons de assets/sons (DMA sur CPC+, lecteur logiciel sur CPC)
gx_sprite(n, x, y, image), gx_sprite_hide(n), gx_sprites_update()	toutes	sprites portables : x en pixels du mode écran ; matériels sur CPC+, logiciels sur CPC
gx_spr_set(n, x, y, image, zoom), gx_spr_hide(n), gx_spr_hide_all(), gx_spr_update()	CPC+, GX4000	sprites matériels : x en pixels mode 2, y signé (un sprite peut sortir par le haut), zoom ZOOM_X1Y1… ZOOM_X4Y4
gx_spr_update_from(premier) ; spr_want[16]	CPC+, GX4000	(0.5.1) recopie seulement les sprites premier..15 : les autres sont gérés par les rasters (multiplexage) ; spr_want = image voulue par sprite
gx_raster_start(table) ; RASTER_INK_ENTRY, RASTER_ANY_ENTRY, RASTER_END_ENTRY ; raster_idle	CPC+, GX4000	interruptions raster : table de couleurs écrite en C
gx_raster_queue(table) ; RASTER_BAND_ENTRY(bloc, ligne),	CPC+, GX4000	(0.5.1) double tampon : la table servira à partir de l'image suivante ; une bande

GX_BAND_SIZE, RASTER_ARM_LINE		recopie X, Y, zoom des sprites 0-5 et 4 couleurs (multiplexage, voir Stellar Wings C)
GX_ASIC_PAL[i], GX_ASIC_SPR_PAL[i]	CPC+, GX4000	(0.5.1) écrire directement une couleur de l'ASIC (&0GRB)
gx_clear_lines(y, n), gx_blit(image, largeur, hauteur, x_octet, y)	toutes	(0.5.1) effacer des lignes ; afficher une image de assets/images (IMG_NOM, IMG_NOM_W, IMG_NOM_H)
gx_sound_tempo(vitesse)	CPC+, GX4000	(0.5.1) change la vitesse du son en cours sur le canal 0 (accélération d'une musique)
gx_sw_set(n, x_octet, y, image), gx_sw_hide(n), gx_sw_update()	CPC	sprites logiciels (x en octets)
gx_tiles_use, gx_tile_draw, gx_map_origin, gx_map_draw, gx_map_get, gx_tile_solid	toutes	décors en tuiles et cartes
gx_anim_start(&etat, ANIM_NOM), gx_anim_step(&etat)	toutes	animations de sprites (gx_anim_t)
gx_rand()	toutes	nombre pseudo-aléatoire 0-255

Les ressources sont visibles dans **build/gen.h** : numéros d'images `F_NOM`, sons `NOM_CH`, `NOM_OFS`, `NOM_SPEED` (utilisés par `GX_SOUND(NOM)`), palettes `pal_screen` et `pal_sprites`, tuiles, cartes, animations. **build/config.h** donne la configuration : `ASIC`, `SCREEN_MODE`, `HAS_KEYBOARD`, `TARGET_GX4000`… pour écrire un seul source pour CPC et CPC+ :

```
#if ASIC
    gx_spr_set(0, x * 2, y, F_VAISSEAU, ZOOM_X2Y1);    /* sprite materiel */
#else
    gx_sw_set(0, x / 2, y, F_VAISSEAU);                /* sprite logiciel */
#endif
```

Une table raster s'écrit comme un simple tableau constant :

```
static const uint8_t raster_bricks[] = {
    BR_TOP - 2,                                        /* un peu avant la brique */
    RASTER_INK_ENTRY(0x00F2, BR_TOP - 1 + 1 * BR_H),   /* rouge (&0GRB) */
    RASTER_INK_ENTRY(0x08F0, BR_TOP - 1 + 2 * BR_H),   /* orange */
    RASTER_INK_ENTRY(0x0FF0, BR_TOP - 1 + 3 * BR_H),   /* jaune */
    RASTER_INK_ENTRY(0x0F40, BR_TOP - 1 + 4 * BR_H),   /* vert */
    RASTER_INK_ENTRY(0x0F0F, BR_TOP - 1 + 5 * BR_H),   /* cyan */
    RASTER_INK_ENTRY(0x044F, BR_TOP - 1 + 6 * BR_H),   /* bleu */
    RASTER_INK_ENTRY(0x0FFF, 199),                     /* retour au blanc (score) */
    RASTER_END_ENTRY                                   /* fin d'image, armement */
};
```

6.6 Mélanger C et assembleur

Les fichiers .asm de src/ s'incluent dans main.asm (`include "outils.asm"`), à côté de la liaison lib/c/gx_c.asm. Les règles sont celles de SDCC :

- Un nom C `nom` s'appelle **_nom** en assembleur (fonction ou variable globale).
- Avec `__sdcccall(0)`, les paramètres sont sur la pile à partir de (SP+2), dans l'ordre : un octet pour un uint8_t, deux pour un uint16_t ou un pointeur. Le résultat revient dans **L** (8 bits) ou **HL** (16 bits). C'est l'appelant qui retire les paramètres.
- L'assembleur peut modifier AF, BC, DE, HL ; il doit **conserver IX** (pointeur de pile du C). **IY** n'est pas utilisé par le C (option --reserve-regs-iy) : il reste libre pour la bibliothèque et vos routines.

Une routine en assembleur appelée depuis le C :

```
/* dans main.c */
uint8_t add3(uint8_t a, uint8_t b, uint8_t c) __sdcccall(0);
```

```
    ...
    total = add3(1, 2, 3);

; dans src/outils.asm (inclus par main.asm)
_add3:  ld hl,2
        add hl,sp               ; HL -> premier parametre
        ld a,(hl)
        inc hl
        add a,(hl)
        inc hl
        add a,(hl)
        ld l,a                  ; resultat 8 bits dans L
        ret
```

Une fonction C appelée depuis l'assembleur, et une variable C lue par l'assembleur :

```
/* dans main.c */
uint8_t double_it(uint8_t v) __sdcccall(0) { return v * 2; }
uint8_t niveau;                 /* globale : _niveau en assembleur */

; en assembleur
        ld a,7
        push af
        inc sp                  ; un parametre 8 bits = 1 octet sur la pile
        call _double_it         ; resultat dans L
        inc sp
        ld a,(_niveau)
```

Quelques lignes d'assembleur peuvent aussi être placées directement dans une fonction C, entre `__asm` et `__endasm;`. Elles suivent alors la syntaxe de SDCC : valeurs précédées de `#`, hexadécimal en `0x` (`ld a,#0x2A`), noms C avec leur _.

Les fonctions de lib/c/gx_c.asm sont de bons modèles : `GX_ENTER` (push ix, IX = SP) puis les paramètres en (ix+4), (ix+5)…, appel de la routine, `GX_LEAVE`. Pour ajouter à l'API une routine de la bibliothèque, il suffit d'y écrire une fonction de ce genre et sa déclaration dans gx.h.

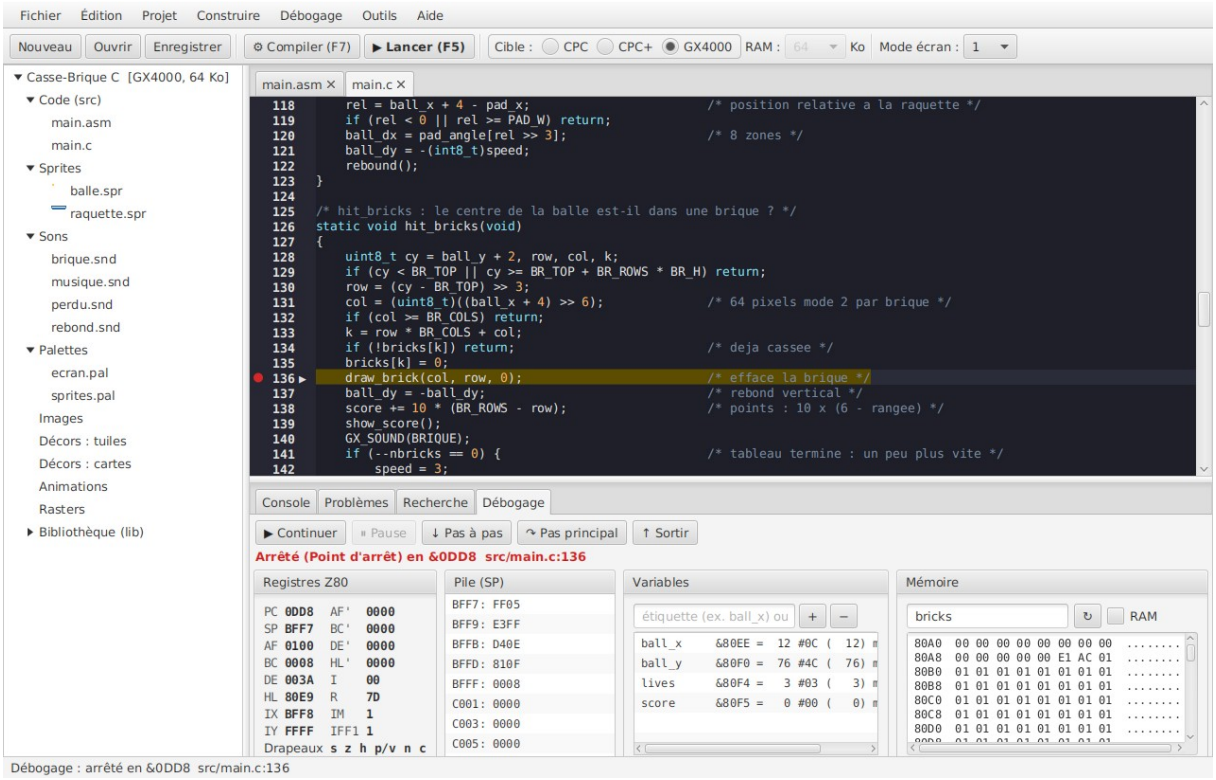
6.7 Erreurs et débogage

Les erreurs de SDCC s'affichent dans l'onglet **Problèmes** comme celles de rasm (double-clic : le fichier .c s'ouvre à la ligne) :

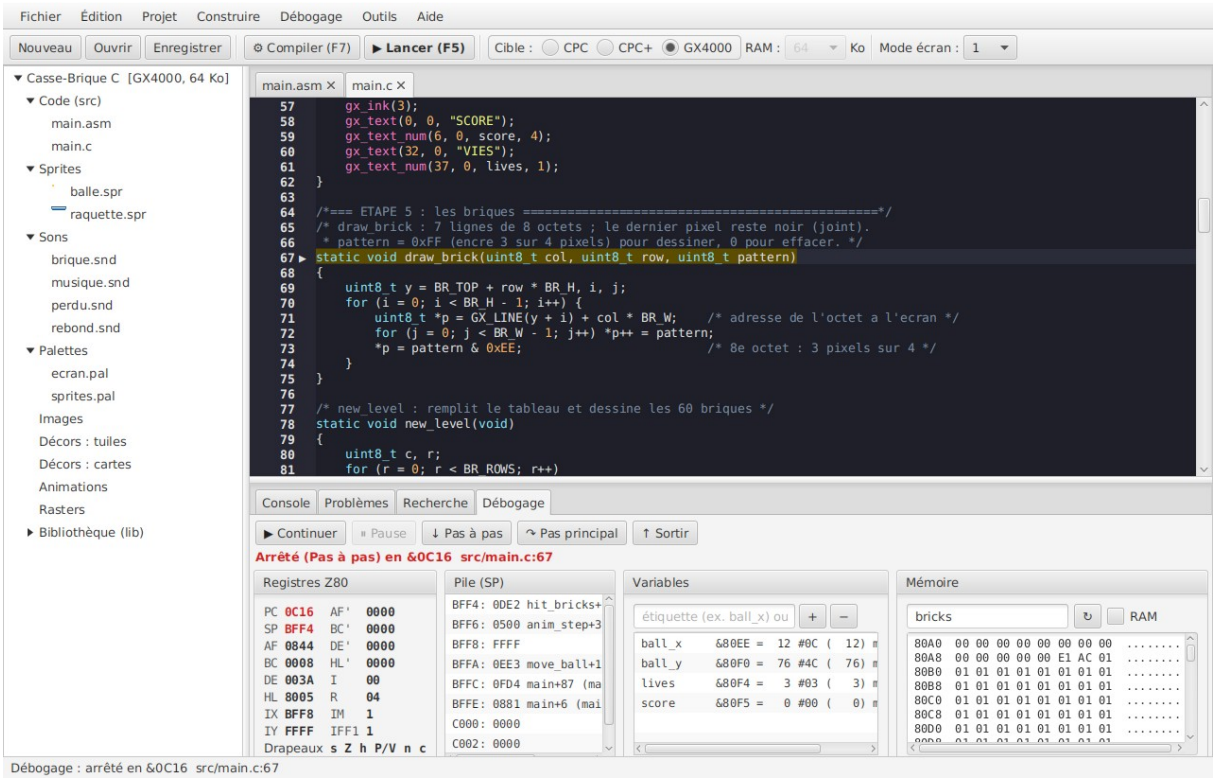
```
== C (SDCC)
  src/main.c:151: error 20: Undefined identifier 'lifes'
== ECHEC (C)
```

Une fonction appelée par le C mais définie nulle part (ni en C, ni dans la bibliothèque de SDCC, ni en assembleur sous le nom _nom) est signalée par « fonction ou variable introuvable », avec le nom attendu en assembleur.

Le débogage fonctionne sur les lignes C exactement comme sur les lignes d'assembleur (Caprice32 corrigé) : **F9** pose un point d'arrêt sur une ligne .c, la ligne courante est surlignée, **F11** avance d'**une ligne C** (et entre dans les fonctions), **F10** passe par-dessus un appel, **Maj+F11** termine la fonction. Dans **Variables**, on tape le nom C (`lives`, `ball_x`), même pour une variable `static` ; la pile affiche les fonctions C (hit_bricks+12, move_ball+19…).



Point d'arrêt sur une ligne C : registres, variables C, tableau bricks en mémoire



F11 : on entre dans draw_brick ; la pile montre hit_bricks, move_ball et main

6.8 Exemple complet

exemples/StellarWingsC est un shoot'em up complet en C (sprites multiplexés par rasters, son DMA, deux joueurs), avec deux routines en assembleur choisies grâce au profil ; sa documentation et son tutoriel sont dans son dossier doc/.

exemples/CasseBriqueC est le casse-brique du tutoriel entièrement réécrit en C (sprites matériels, briques dessinées dans l'écran, rasters, sons DMA, score). Il suit les mêmes étapes que

la version en assembleur (exemples/CasseBrique), ce qui permet de comparer les deux ; le tutoriel lui consacre un chapitre.



Le casse-brique en C dans Caprice32

7. Le code Java

GX Studio est écrit en Java 17 avec JavaFX ; il ne dépend que de RichTextFX (éditeur de code). Le code est découpé en quatre paquetages, faciles à étendre (nouvel éditeur, nouveau format de ressource, autre émulateur).

Classe	Rôle
model.Target	les trois machines : ASIC, clavier, mémoires possibles, format de sortie
model.Project	le projet (projet.gxs) et l'accès aux ressources
model.Sprite, model.Palette	formats .spr et .pal ; conversions de couleurs (27 couleurs CPC, mot ASIC &0GRB)
gen.AssetCompiler	ressources -> build/ (config.inc, gen.asm, sprites, images, banques, save.inc)
gen.SoundCompiler	fichiers .snd -> listes DMA
gen.AySynth	émulation du PSG pour l'écoute des sons dans le studio
build.Builder	copie des sources, points d'arrêt, appel de rasm, lecture des erreurs et des symboles
build.CCompiler, build.SdccToRasm	projets en C : appel de SDCC, fonctions de sa bibliothèque à la demande, traduction sdas -> rasm
build.Emulator	ligne de commande et lancement de Caprice32 (dossier, ROM, port du débogueur)
build.DebugClient	liaison avec le serveur de débogage de Caprice32 (commandes, arrêts, lecture mémoire)
build.ProjectFactory, build.Settings	création des projets à partir des modèles ; réglages
ui.StudioApp	fenêtre principale, menus, arbre du projet, compilation en tâche de fond
ui.AsmEditor, SpriteEditor, PaletteEditor, SoundEditor, ImageViewer	les onglets d'édition (classe commune EditorTab)
ui.DebugPanel	onglet Débogage : commandes, registres, pile, variables, mémoire
Cli	commandes new / build / run sans interface

```
    public static List<String> command(Project p, Settings s) { return command(p, s, 0); }
```

```
    /** debugPort > 0 : active le serveur de debogage du Caprice32 corrige (ignore par un
Caprice32 d'origine). */
    public static List<String> command(Project p, Settings s, int debugPort) {
        List<String> c = new ArrayList<>();
        c.add(s.caprice);
        int model = model(p);
        int ram = model >= 2 ? Math.max(128, p.ramKb) : 64;
        if (p.target == Target.GX4000) ram = 128;       // Caprice32 d'origine : 128 Ko
minimum en 6128+
        c.addAll(List.of("-O", "system.model=" + model, "-O", "system.ram_size=" + ram));
        // Caprice32 cherche ses ROM (cpc464/664/6128, amsdos) dans ./rom du dossier COURANT
:
        // lance depuis le studio, il ne les trouve pas et quitte aussitot en CPC (la
cartouche
        // CPC+/GX contient sa propre ROM, d'ou un plantage seulement pour les cibles CPC).
        Path roms = romDir(s);
        if (roms != null) c.addAll(List.of("-O", "rom.rom_path=" + roms));
        if (debugPort > 0) c.addAll(List.of("-O", "devtools.server_port=" + debugPort,
                "-O", "system.auto_pause=0"));                  // le focus ne doit pas
relancer un programme arrete
        Path sym = p.build().resolve("caprice.sym");
```

8. Limites de cette version et suite possible

- Défilement 8 directions : sur CPC il avance par caractère et par rangée de 8 lignes (le CPC n'a pas de soft-scroll) ; un défilement plus fin y demanderait des techniques de « rupture » du CRTC.
- L'import MP3 reste une approximation : pour un résultat propre, partir d'un MIDI du morceau quand il existe, puis retoucher la grille.
- Les rasters du studio changent des encres ; les autres effets de l'ASIC (couleurs des sprites par bande, multiplexage, écran partagé pour un panneau fixe pendant le défilement) restent à programmer avec raster.asm.
- C : un seul modèle mémoire (code dans la page 0 avec l'assembleur) ; pas de bancs de code C ni de float optimisé. Les grosses données peuvent aller dans d'autres pages, en assembleur.
- Débogueur : pas de points d'arrêt conditionnels ni de surveillance d'écriture en mémoire depuis le studio (Caprice32 les propose dans ses propres outils, Maj+F2).